

Nora

Conversational Voice Agent for Automated Payment and Service Reminders

FYP1 Final Report

Project Code: S26-009-D

Project Team

Ehsan Umran (22i-2724)

Ayesha Suleman (22i-2002)

Khadeejah Alam (22i-2063)

Session 2022–2026

Supervised by

Dr. Muhammad Nouman Noor

Department of Software Engineering

National University of Computer and Emerging Sciences

Islamabad, Pakistan

June, 2026

Abstract

This report presents the final documentation for Nora, an AI-powered conversational voice agent designed to automate payment and service reminders for various service providers. The system initiates outbound calls using the Twilio telephony API, processes spoken user responses through the Whisper speech-to-text engine, detects intent using Groq-powered large language model inference, generates context-aware replies through ElevenLabs text-to-speech synthesis, and logs all interactions to Supabase for reporting and analytics.

The focus of this project is to develop a functional prototype demonstrating the key capabilities of automated two-way voice communication: reducing the workload of manual call centres while improving customer experience through personalised, context-aware interactions. The system operates on synthetic data and does not process real financial transactions. Future enhancements may include multi-language support beyond English and Urdu, advanced sentiment analysis, predictive analytics, and full-scale enterprise deployment capabilities.

This document follows the same structure as the FYP1 Final Report template, covering all four system modules – AI Voice Agent, Speech Processing, Conversation Management, and Admin Dashboard – with complete use-cases, functional and non-functional requirements, system architecture, design models, implementation algorithms, external APIs, and unit testing details.

Table of Contents

Introduction [AS PER SCOPE DOCUMENT]

Customer communication is a critical component for businesses such as Internet Service Providers (ISPs), subscription services, gyms, and utility providers. Traditional reminder mechanisms – including SMS, emails, and manual calls – often fail to achieve high response rates. Users frequently ignore passive notifications, resulting in missed payments, service interruptions, and increased customer complaints. These challenges highlight the need for a more intelligent, interactive, and scalable communication system.

This project proposes Nora, an AI-powered conversational voice agent capable of initiating two-way outbound calls, understanding user responses in real time, detecting intent, and providing context-aware replies. By automating routine reminders, Nora aims to improve operational efficiency, reduce manual workload for call centres, and enhance customer satisfaction through personalised, natural-sounding interactions.

The system leverages state-of-the-art technologies including the Twilio Programmable Voice API for telephony, Whisper by OpenAI for speech-to-text transcription, ElevenLabs for text-to-speech synthesis, and Groq for high-performance large language model inference powering intent and sentiment classification. Data persistence and authentication are managed via Supabase (PostgreSQL), while the admin dashboard is built on Next.js with React and Tailwind CSS.

Prior research in conversational AI and automated reminder systems highlights significant gaps in existing solutions: most rely on one-way notifications lacking two-way dialogue, cannot handle user intent dynamically, and fail to escalate complex cases to human agents. Nora addresses all these gaps with a modular, extensible architecture grounded in modern AI and telephony integrations.

This chapter provides the foundation for understanding the Nora system, outlining the background context, examining current limitations, defining the problem space, and clarifying the scope and modular structure of the development effort.

1.1 Existing Solutions

Several automated communication systems and voice bot platforms currently exist across the industry, including basic IVR (Interactive Voice Response) systems, SMS-based reminder services, and proprietary call-centre automation tools. These systems generally offer one-directional notifications, predefined script-based interactions, and limited natural language understanding.

Existing solutions typically include simple outbound diallers, SMS notification services, or rigid IVR menus that require users to press numeric keys rather than speak naturally. While these demonstrate the value of automated communication, they fall short in key areas:

- Most lack two-way spoken dialogue, meaning they cannot interpret or respond to natural user speech.
- Few support real-time intent classification – the ability to understand whether a customer is confirming payment, requesting a follow-up, or indicating a wrong number.
- Many do not include escalation logic to transfer complex or unresolved calls to human agents.

- Existing tools rarely provide a fully integrated admin dashboard with call logging, transcript viewing, analytics, and campaign management in a single interface.
- Almost none support multi-language voice interactions (e.g., both English and Urdu) within the same system.

These limitations highlight the need for a unified, intelligent, and conversational reminder system that truly automates end-to-end customer communication.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
System 1 – Basic IVR Dialler	Initiates outbound calls and plays pre-recorded audio messages. Requires users to press keys to respond.	No natural language understanding; cannot detect spoken intent or adapt responses. No escalation logic.
System 2 – SMS Reminder Service	Sends automated text messages to customers with payment due dates and link-based confirmation.	One-way only; frequently ignored. No voice interaction, no intent detection, no dynamic rescheduling.
System 3 – Manual Call Centre	Human agents make outbound reminder calls to customers, record outcomes manually, and schedule follow-ups.	Highly resource-intensive and expensive. Inconsistent quality, limited scalability, high error rates.

1.2 Problem Statement

Existing reminder systems face several critical limitations. First, SMS and email notifications are frequently ignored, leading to delayed or missed payments that disrupt business cash flow and service continuity. Second, manual call centres are resource-intensive, increasing operational costs and adding significant delays at scale. Third, current automated systems lack genuine two-way communication, preventing real-time confirmation of customer intent or dynamic handling of exceptions such as rescheduling requests or wrong-number detection. Finally, abrupt service interruptions resulting from missed reminders frustrate customers and damage business reputation.

The core problem is the absence of an accessible, scalable, and intelligent automated voice communication platform that supports natural two-way spoken dialogue, real-time intent classification, multi-language support, and integrated analytics – all within a single deployable system. Nora directly addresses this gap by providing a complete AI-driven voice agent that automates the entire outbound reminder workflow from call initiation through intent resolution to outcome logging.

1.3 Scope

The scope of Nora defines the functional boundaries, objectives, and limitations of the system. The project focuses on developing a working prototype of an AI-based conversational voice agent that can handle the essential aspects of automated payment and service reminders. The main functionality includes making outbound voice calls to customers via Twilio, understanding their spoken responses through Whisper-based speech processing, detecting intent using Groq LLM inference, and responding in a way that simulates a natural conversation.

The system additionally includes a conversation management component to maintain contextual coherence across multi-turn dialogues, and an admin dashboard where call logs, transcripts, and analytics can be viewed and monitored. Multi-language support for English and Urdu is included in the prototype scope.

This prototype operates on synthetic data and will not involve actual financial transactions or real customer data. It is not designed for large-scale enterprise deployment in this phase. Several areas fall outside the current scope: real payment gateway integration, supply chain or business-system integration, advanced dialect variations beyond standard English and Urdu, and AI-based predictive analytics. These may be incorporated in future development phases.

1.4 Modules

This section provides an overview of the four key system modules that collectively form the Nora platform. Each module supports a distinct set of responsibilities, ensuring modularity, maintainability, and a clear architectural structure.

1.4.1 Module 1: AI Voice Agent

The core module responsible for all customer call interactions. It initiates outbound calls via Twilio, delivers personalised reminder messages via TTS, captures and processes user responses, manages call flow (confirmation, retry, termination), escalates to human agents when necessary, and logs all call metadata for reporting.

1.4.2 Module 2: Speech Processing

Handles bidirectional conversion between spoken language and text. Captures user voice input and converts it to text using Whisper STT. Generates natural-sounding audio responses using ElevenLabs TTS. Supports multi-language processing with language detection and basic noise handling.

1.4.3 Module 3: Conversation Management

Ensures interactions remain context-aware and consistent throughout the call. Maintains conversation state and tracks previous user responses. Handles different conversation paths (payment confirmation, rescheduling, wrong-number detection, escalation). Incorporates sentiment awareness to adapt responses when users appear confused or unresponsive.

1.4.4 Module 4: Admin Dashboard

Provides a web-based interface for administrators to monitor and manage all system activity. Displays call logs, conversation transcripts, detected intents, success/failure rates, and analytics. Supports CSV export of reports, campaign scheduling, and role-based access management.

1.5 Work Division

Effective division of work is critical for ensuring that the development of Nora proceeds in a structured, efficient, and goal-oriented manner. Given the multidimensional nature of the system – spanning telephony integration, AI inference, speech processing, frontend development, backend APIs, and deployment – the team adopted a collaborative yet role-specialised approach. This strategy allowed each member to focus on core competencies while contributing collectively to cross-cutting tasks such as testing, documentation, and integration.

The work division also ensured parallel development across modules, reducing bottlenecks and enabling iterative refinement throughout the project lifecycle. Responsibilities were

assigned to maintain balance between technical and analytical tasks, while promoting accountability and structured progress monitoring.

1.5.1 Individual Responsibilities

Ehsan Umran – Backend & Integration: Ehsan was responsible for the backend API development and system integration. His work included designing the FastAPI-based server architecture, managing the Supabase database schema, integrating all system modules including the Twilio telephony layer and the admin dashboard, and ensuring secure and reliable data flow across all components. He also oversaw CI/CD pipeline configuration and cloud deployment.

Ayesha Suleman – Conversation Flow & Compliance: Ayesha was responsible for designing and implementing the structured conversation flows and intent-based dialogue handling. Her work included defining the conversation state machine, implementing compliance rules such as Do-Not-Call enforcement and call frequency limits, designing the escalation logic for handling complex or unresolved interactions to human agents, and ensuring contextual continuity across multi-turn dialogues.

Khadeejah Alam – Speech Processing & AI: Khadeejah was responsible for implementing the speech processing pipeline. Her work included integrating Whisper STT and ElevenLabs TTS modules, developing the intent and sentiment detection models via Groq inference, improving speech recognition accuracy through noise handling and confidence-score filtering, and collaborating with conversation management to maintain context-aware interactions.

1.5.2 Collaborative Responsibilities

Beyond individual responsibilities, all team members participated in core collaborative activities essential to the success of the project:

- **System Architecture Design:** All members jointly designed the modular architecture, agreed on API interfaces between components, and validated the technology stack selection.
- **Comprehensive Project Documentation:** All members contributed to writing, editing, and maintaining documentation including the mid-report, final report, use-case specifications, and system manuals.
- **System Validation and Quality Assurance:** Testing was performed collaboratively to validate call flow logic, intent classification accuracy, speech recognition quality, and overall system reliability.
- **Dashboard UX Review:** Group review sessions were held to evaluate admin interface usability, analytics visualisation clarity, and call log readability.
- **CI/CD Pipeline and Deployment:** The team jointly configured and monitored automated deployment pipelines on cloud virtual machines, ensuring smooth and reproducible deployments.

Table 1.2: Work Division Overview

Team Member	Responsibilities
Ehsan Umran	Backend API development; FastAPI server architecture; Supabase schema design; Twilio and dashboard integration; CI/CD and cloud deployment management.
Ayesha Suleman	Conversation flow design; intent-based dialogue implementation; compliance rules enforcement (DNC, call limits); escalation logic and state machine design.

Team Member	Responsibilities
Khadeejah Alam	Whisper STT and ElevenLabs TTS integration; intent and sentiment detection via Groq; noise handling; speech recognition accuracy improvements.
Collaborative Tasks	System architecture; documentation; testing and validation; dashboard UX review; CI/CD pipeline setup; deployment activities.

Project Requirements [AS PER FYP1 MID REPORT]

This chapter describes the functional and non-functional requirements of the Nora system. It outlines the modular design where each module focuses on a specific set of responsibilities to ensure scalability, maintainability, and structured development. The modular breakdown also helps distribute work efficiently among the team and supports future extensibility.

2.1 Use-Cases

2.1.1 Module 1 – AI Voice Agent Module

Use Case ID	UC-1.1
Use Case Name	Initiate Outbound Reminder Call
Actor(s)	System
Stakeholders & Interests	Service providers want automated reminder calls to reduce manual effort while ensuring timely communication with customers.
Trigger	Scheduled reminder time is reached for a customer record.
Preconditions	Customer record exists in the database and a reminder call is scheduled.
Success Postconditions	Call connects and the reminder message is delivered to the customer.
Failure Postconditions	Call attempt fails and retry logic is scheduled automatically.
Priority	High
Complexity	Moderate
Frequency of Use	High
Functional Requirement(s) Reference	FR-1.1.1, FR-1.1.2, FR-1.1.3
Main Flow	1. System retrieves the scheduled customer record from the database. 2. System initiates an outbound call via the Twilio telephony API. 3. System delivers the pre-defined personalised reminder message via TTS.
Alternative Flows	1. If the call is unanswered, retry is scheduled according to the retry policy.
Exceptions	1. Network failure or telephony API unavailability.
Special Requirements	Must support real-time audio streaming via Twilio in test mode.
Assumptions	Twilio telephony service is configured and available.
Notes and Issues	Call cost optimisation and SMS fallback may be added in future versions.

Table 3.25: Use Case – Initiate Outbound Reminder Call

Use Case ID	UC-1.2
Use Case Name	Retry Failed Call

Actor(s)	System
Stakeholders & Interests	Service providers require automatic retries to maximise contact rate and reduce missed reminders.
Trigger	An outbound call attempt fails (unanswered, busy, or network error).
Preconditions	A retry policy is enabled for the customer record.
Success Postconditions	Call connects successfully during a retry attempt.
Failure Postconditions	Maximum retry attempts are reached and reminder is marked as failed.
Priority	High
Complexity	Moderate
Frequency of Use	Frequent
Functional Requirement(s) Reference	FR-1.1.4, FR-1.8.1, FR-1.8.3
Main Flow	1. System detects a failed call attempt and logs the failure reason. 2. System schedules a retry based on the configured retry interval. 3. System attempts the call again at the scheduled retry time.
Alternative Flows	1. Customer answers during a retry and conversation proceeds normally.
Exceptions	1. Retry scheduler service failure.
Special Requirements	Retry interval must be configurable per campaign.
Assumptions	Telephony service supports multiple call attempts.
Notes and Issues	SMS fallback channel may be added as a future enhancement.

Table 3.26: Use Case – Retry Failed Call

Use Case ID	UC-1.3
Use Case Name	Escalate Call to Human Agent
Actor(s)	System, Human Agent
Stakeholders & Interests	Complex or ambiguous cases must be handled by human agents to ensure customer satisfaction and issue resolution.
Trigger	An escalation rule is triggered (e.g., intent confidence below threshold or multiple unrecognised responses).
Preconditions	An active call session is in progress.
Success Postconditions	The call is transferred successfully to a human agent with conversation context.
Failure Postconditions	Escalation fails; call is logged and a follow-up is scheduled.
Priority	High
Complexity	Moderate

Frequency of Use	Low
Functional Requirement(s) Reference	FR-1.3.1, FR-1.3.2, FR-1.3.3
Main Flow	1. System detects an escalation condition during the active call. 2. System transfers the call to an available human agent. 3. System provides the agent with conversation history and detected intents.
Alternative Flows	1. If no agent is available, system schedules a callback.
Exceptions	1. Call transfer failure due to telephony API error.
Special Requirements	Conversation context must be preserved and passed during transfer.
Assumptions	Human agents are available during configured working hours.
Notes and Issues	CRM integration for agent context may be added in future versions.

Table 3.27: Use Case – Escalate Call to Human Agent

Use Case ID	UC-1.4
Use Case Name	Manage Conversation Flow
Actor(s)	System
Stakeholders & Interests	Service providers need consistent, context-aware dialogue management to ensure calls are resolved efficiently and correctly.
Trigger	An active call session is in progress.
Preconditions	A call has connected and user input is being captured.
Success Postconditions	Conversation follows the correct path (confirmation, rescheduling, or escalation) and terminates politely.
Failure Postconditions	Conversation path is unclear; retry logic is triggered.
Priority	High
Complexity	High
Frequency of Use	Continuous (every call)
Functional Requirement(s) Reference	FR-1.4.1, FR-1.4.2, FR-1.4.3
Main Flow	1. System initialises and maintains the conversation state throughout the call. 2. System stores previous user responses for contextual continuity. 3. System routes to the appropriate conversation path based on detected intent.
Alternative Flows	1. Unclear input triggers retry logic with simplified rephrasing.
Exceptions	1. State management failure due to unexpected disconnection.
Special Requirements	Conversation state must persist across multi-turn exchanges within a session.
Assumptions	Intent detection module is operational throughout the call.

Notes and Issues	Advanced multi-session context may be explored in future versions.
-------------------------	--

Table 3.28: Use Case – Manage Conversation Flow

Use Case ID	UC-1.5
Use Case Name	Process Payment Confirmation
Actor(s)	Customer, System
Stakeholders & Interests	Service providers require verbal confirmation of payment to update records and avoid unnecessary follow-ups.
Trigger	Customer verbally confirms that a payment has been made.
Preconditions	An active reminder call is in progress.
Success Postconditions	Payment status is marked as confirmed in the database (synthetic data only) and the call is closed.
Failure Postconditions	Intent is not recognised with sufficient confidence; system requests clarification.
Priority	High
Complexity	Moderate
Frequency of Use	High
Functional Requirement(s) Reference	FR-1.5.1, FR-1.5.3, FR-1.5.4
Main Flow	1. System detects a payment confirmation intent from the user's response. 2. System updates the reminder/payment status in the database. 3. System verbally acknowledges the confirmation and terminates the call.
Alternative Flows	1. Low-confidence detection triggers a clarification request.
Exceptions	1. Database update failure.
Special Requirements	Intent confidence threshold must be met before confirming. System operates on synthetic data only in prototype.
Assumptions	Customer provides a clear verbal response.
Notes and Issues	Real payment gateway integration may be added in future production versions.

Table 3.29: Use Case – Process Payment Confirmation

Use Case ID	UC-1.6
Use Case Name	Reschedule Reminder
Actor(s)	Customer, System
Stakeholders & Interests	Customers may request additional time before payment; the system should flexibly accommodate this to reduce hostility.
Trigger	Customer requests a new reminder date during the call.
Preconditions	An active call session is in progress.

Success Postconditions	The new reminder schedule is stored in the database and confirmed verbally to the customer.
Failure Postconditions	Invalid date/time is rejected and user is asked to provide a valid alternative.
Priority	High
Complexity	Moderate
Frequency of Use	Moderate
Functional Requirement(s) Reference	FR-1.6.1, FR-1.6.3, FR-1.6.4
Main Flow	1. System detects a rescheduling intent from the user's response. 2. System asks for and extracts the preferred date and time from user input. 3. System validates the date/time, stores the new schedule, and confirms verbally.
Alternative Flows	1. Invalid or unclear date format prompts a correction request.
Exceptions	1. Scheduler service failure.
Special Requirements	System must validate time conflicts and reasonable future dates.
Assumptions	Customer provides a valid and clear date/time.
Notes and Issues	SMS or dashboard confirmation of rescheduling may be added later.

Table 3.30: Use Case – Reschedule Reminder

Use Case ID	UC-1.7
Use Case Name	Log and Store Call Records
Actor(s)	System
Stakeholders & Interests	Administrators need complete call records for monitoring, auditing, and improving system performance.
Trigger	A call session ends (successfully, failed, or escalated).
Preconditions	A call session has been initiated.
Success Postconditions	Complete call metadata, transcripts, and outcome are stored in the database.
Failure Postconditions	Log storage fails and an error is flagged for the administrator.
Priority	High
Complexity	Low
Frequency of Use	Every call
Functional Requirement(s) Reference	FR-1.7.1, FR-1.7.3, FR-1.7.5
Main Flow	1. System records metadata for the call (duration, timestamp, status, outcome). 2. System stores the full call transcript generated during the

	session. 3. System associates the log with the corresponding customer record.
Alternative Flows	1. Partial log saved if call was interrupted mid-session.
Exceptions	1. Database connection error during log write.
Special Requirements	Logs must be securely archived and accessible via the admin dashboard.
Assumptions	Supabase database is connected and available.
Notes and Issues	Log retention policy and archiving strategy to be defined in future phases.

Table 3.31: Use Case – Log and Store Call Records

Use Case ID	UC-1.8
Use Case Name	Detect and Handle Call Failures
Actor(s)	System
Stakeholders & Interests	Service providers need assurance that failed calls are detected, logged, and handled systematically.
Trigger	A call attempt results in an error (busy signal, network failure, no answer, unreachable).
Preconditions	A call has been initiated.
Success Postconditions	Failure reason is logged and retry logic or failure status is applied.
Failure Postconditions	Failure detection itself fails; call remains in an ambiguous state.
Priority	High
Complexity	Moderate
Frequency of Use	Frequent
Functional Requirement(s) Reference	FR-1.8.1, FR-1.8.2, FR-1.8.3
Main Flow	1. System detects the call failure and identifies the failure reason. 2. System logs the failure reason linked to the customer record. 3. System applies the retry policy or marks the reminder as permanently failed.
Alternative Flows	1. Admin is notified if failure threshold for a campaign is exceeded.
Exceptions	1. Telephony API does not return a clear failure code.
Special Requirements	All failure types (busy, no answer, unreachable) must be distinctly classified.
Assumptions	Twilio returns standard failure codes.
Notes and Issues	Failure analytics can be extended to the admin dashboard.

Table 3.32: Use Case – Detect and Handle Call Failures

Use Case ID	UC-1.9
Use Case Name	Personalise Voice Interaction
Actor(s)	System
Stakeholders & Interests	Customers are more responsive to personalised messages; service providers want higher engagement rates.
Trigger	System prepares to initiate or respond during a call.
Preconditions	Customer profile data is available in the database.
Success Postconditions	Personalised reminder message is delivered using the customer's name, account details, and preferred language.
Failure Postconditions	Personalisation data unavailable; system falls back to a generic message.
Priority	High
Complexity	Moderate
Frequency of Use	Every call
Functional Requirement(s) Reference	FR-1.9.1, FR-1.9.4, FR-1.9.5
Main Flow	1. System retrieves customer-specific data (name, amount, due date, language preference). 2. System generates a personalised reminder message using the customer's details. 3. System delivers the personalised message in the appropriate language via TTS.
Alternative Flows	1. Missing data fields are replaced with generic placeholders.
Exceptions	1. Language model unavailable for the preferred language.
Special Requirements	Must support English and Urdu personalised messages in the prototype.
Assumptions	Customer records include language preference and relevant account details.
Notes and Issues	Additional language support and dynamic template editor to be added in future.

Table 3.33: Use Case – Personalise Voice Interaction

Use Case ID	UC-1.10
Use Case Name	Enforce Call Compliance Rules
Actor(s)	System
Stakeholders & Interests	Businesses must comply with Do-Not-Call regulations and call frequency restrictions to avoid legal penalties.
Trigger	A call is about to be initiated for a customer.
Preconditions	Customer record and compliance settings are available.
Success Postconditions	Call is either placed within compliant parameters or blocked and logged.

Failure Postconditions	Compliance check fails; call proceeds without verification (error logged).
Priority	High
Complexity	Moderate
Frequency of Use	Every call
Functional Requirement(s) Reference	FR-1.10.1, FR-1.10.2, FR-1.10.4
Main Flow	1. System checks whether the current time is within permitted calling hours. 2. System checks whether the customer is on the Do-Not-Call list or has opted out. 3. If compliant, call proceeds. If not, call is blocked and reason is logged.
Alternative Flows	1. Customer requests opt-out during a call; system terminates and updates the DNC list.
Exceptions	1. Compliance configuration file is missing or corrupt.
Special Requirements	System must immediately terminate calls if a live opt-out request is detected.
Assumptions	DNC list is maintained and up to date in the database.
Notes and Issues	Regional regulation variations may require future enhancements.

Table 3.34: Use Case – Enforce Call Compliance Rules

Use Case ID	UC-1.11
Use Case Name	Monitor AI Performance Metrics
Actor(s)	System, Admin
Stakeholders & Interests	Administrators and the development team need insight into AI accuracy, call outcomes, and overall system quality for improvement.
Trigger	Admin opens the analytics or performance section of the dashboard.
Preconditions	Sufficient call records and logs are available in the database.
Success Postconditions	Performance metrics are displayed including call success rate, intent accuracy, and average duration.
Failure Postconditions	Data unavailable or incomplete; dashboard shows partial or default metrics.
Priority	Medium
Complexity	Moderate
Frequency of Use	Frequent
Functional Requirement(s) Reference	FR-1.11.1, FR-1.11.3, FR-1.11.4
Main Flow	1. System retrieves call records and aggregates performance data. 2. System calculates call success rate, intent recognition accuracy, and average duration. 3. Dashboard displays the performance summary with charts and metrics.

Alternative Flows	1. Admin filters metrics by date range or campaign.
Exceptions	1. Visualisation rendering error or analytics engine timeout.
Special Requirements	Dashboard must display call success rate and escalation statistics.
Assumptions	Analytics engine and Supabase are operational.
Notes and Issues	Advanced predictive analytics may be added in future versions.

Table 3.35: Use Case – Monitor AI Performance Metrics

2.1.2 Module 2 – Speech Processing Module

Use Case ID	UC-2.1
Use Case Name	Convert Speech to Text
Actor(s)	System
Stakeholders & Interests	Accurate speech recognition is fundamental for correct intent detection and effective conversation management.
Trigger	Customer speaks during an active call session.
Preconditions	A call is active and audio is being streamed to the STT engine.
Success Postconditions	User speech is accurately converted to text and passed to the intent detection module.
Failure Postconditions	Recognition fails; system requests the customer to repeat their response.
Priority	High
Complexity	High
Frequency of Use	Continuous (every user utterance)
Functional Requirement(s) Reference	FR-1.2.2, FR-1.9.4
Main Flow	1. System captures the user's voice input from the active audio stream. 2. System processes the audio using the Whisper STT engine. 3. System returns the transcribed text to the conversation management module.
Alternative Flows	1. If input is unclear or below confidence threshold, system requests the user to repeat.
Exceptions	1. Excessive background noise causes misrecognition.
Special Requirements	Must support multi-language input with initial support for English and Urdu.
Assumptions	Customer speaks clearly in a supported language.
Notes and Issues	Whisper model fine-tuning for Urdu accent variations may improve accuracy.

Table 3.36: Use Case – Convert Speech to Text

Use Case ID	UC-2.2
-------------	--------

Use Case Name	Convert Text to Speech
Actor(s)	System
Stakeholders & Interests	The system must generate natural-sounding voice responses to ensure a smooth and engaging customer experience.
Trigger	The conversation management module generates a text response to deliver to the customer.
Preconditions	A text response has been generated by the conversation or intent module.
Success Postconditions	The text is converted into natural-sounding audio and streamed to the customer.
Failure Postconditions	Audio generation fails; fallback voice or a simpler message is used.
Priority	High
Complexity	High
Frequency of Use	Continuous (every system response)
Functional Requirement(s) Reference	FR-1.2.5, FR-1.9.2
Main Flow	1. System receives the text response from the conversation module. 2. System processes the text using the ElevenLabs TTS engine. 3. System streams the generated audio to the customer via the active call.
Alternative Flows	1. If the primary voice is unavailable, system uses a configured fallback voice.
Exceptions	1. Audio streaming interruption due to network instability.
Special Requirements	Voice output must sound natural and clear. Response latency must be minimised.
Assumptions	ElevenLabs TTS API is available and configured.
Notes and Issues	Future enhancement may include emotion-based voice tone modulation.

Table 3.37: Use Case – Convert Text to Speech

Use Case ID	UC-2.3
Use Case Name	Detect Language
Actor(s)	System
Stakeholders & Interests	The system should respond in the language spoken by the customer for better communication and higher engagement.
Trigger	Customer begins speaking at the start of or during a call.
Preconditions	Customer speech has been captured successfully.
Success Postconditions	Language is identified and subsequent responses are adapted to use the detected language.
Failure Postconditions	Detection fails; system defaults to English or the pre-configured campaign language.
Priority	Medium

Complexity	Moderate
Frequency of Use	Once per call session
Functional Requirement(s) Reference	FR-1.9.4, FR-1.9.5
Main Flow	1. System captures the initial speech input from the customer. 2. System analyses linguistic patterns to identify the spoken language. 3. System selects the appropriate language model and TTS voice for the session.
Alternative Flows	1. If confidence is low, system asks the customer to confirm their preferred language.
Exceptions	1. Mixed-language input causes detection uncertainty.
Special Requirements	Prototype must support English and Urdu as the initial two languages.
Assumptions	Language detection API or Whisper language identification is integrated.
Notes and Issues	Future versions may support regional accents and additional languages.

Table 3.38: Use Case – Detect Language

Use Case ID	UC-2.4
Use Case Name	Handle Noise and Improve Recognition Accuracy
Actor(s)	System
Stakeholders & Interests	Accurate transcription despite poor audio conditions is critical for the system to function reliably in real-world environments.
Trigger	Audio input is received with detectable background noise or low signal quality.
Preconditions	The STT module is active and receiving audio input.
Success Postconditions	Noise is filtered sufficiently for the STT engine to produce an accurate transcription.
Failure Postconditions	Noise level is too high; transcription confidence is below threshold and clarification is requested.
Priority	Medium
Complexity	Moderate
Frequency of Use	As needed
Functional Requirement(s) Reference	FR-1.2.2, FR-2.4.1
Main Flow	1. System receives audio stream from the active call. 2. System applies basic noise filtering before passing audio to the STT engine. 3. STT engine transcribes the cleaned audio and returns a confidence score.
Alternative Flows	1. If confidence is below threshold, system requests the user to repeat more clearly.
Exceptions	1. Noise level renders the audio entirely unintelligible.

Special Requirements	System must handle common telephony noise profiles (line static, background speech).
Assumptions	Whisper or equivalent model includes noise-robust recognition.
Notes and Issues	Advanced noise cancellation may be added as a post-processing step.

Table 3.39: Use Case – Handle Noise and Improve Recognition Accuracy

2.1.3 Module 3 – Conversation Management Module

Use Case ID	UC-3.1
Use Case Name	Confirm Payment
Actor(s)	Customer, System
Stakeholders & Interests	Service providers require reliable confirmation of payments to update records and stop follow-up reminders.
Trigger	Customer verbally confirms that a payment has been or will be made.
Preconditions	An active reminder call is in progress and a confirmation intent has been detected.
Success Postconditions	Payment status is marked as confirmed in the system.
Failure Postconditions	Intent not recognised with sufficient confidence; clarification is requested.
Priority	High
Complexity	Moderate
Frequency of Use	High
Functional Requirement(s) Reference	FR-1.5.1, FR-1.5.3, FR-1.5.4
Main Flow	1. System detects a payment confirmation intent from the transcribed response. 2. System updates the payment status record in the database. 3. System verbally acknowledges the confirmation and ends the call.
Alternative Flows	1. Low confidence score triggers a clarification request before updating records.
Exceptions	1. Database update failure prevents status from being recorded.
Special Requirements	Intent detection confidence threshold must be met. System uses synthetic data only.
Assumptions	Customer provides a clear and unambiguous verbal response.
Notes and Issues	Integration with a real payment gateway may be pursued in future versions.

Table 3.40: Use Case – Confirm Payment

Use Case ID	UC-3.2
Use Case Name	Reschedule Reminder (Conversation Layer)

Actor(s)	Customer, System
Stakeholders & Interests	Customers may need additional time before making a payment; flexible rescheduling improves satisfaction and reduces hostility.
Trigger	Customer verbally requests to be called again at a different time.
Preconditions	An active call session is in progress.
Success Postconditions	New reminder date and time are validated and stored; customer is informed.
Failure Postconditions	Invalid or unclear date/time is rejected; user is prompted to try again.
Priority	High
Complexity	Moderate
Frequency of Use	Moderate
Functional Requirement(s) Reference	FR-1.6.1, FR-1.6.3, FR-1.6.4
Main Flow	1. System detects a rescheduling intent from the user's response. 2. System asks for and extracts the preferred date and time. 3. System validates the provided date/time, stores the updated schedule, and confirms.
Alternative Flows	1. Ambiguous date input triggers a follow-up question for clarification.
Exceptions	1. Scheduler service failure prevents the new schedule from being stored.
Special Requirements	System must validate time conflicts and reject dates in the past.
Assumptions	Customer provides a valid and interpretable date/time expression.
Notes and Issues	Confirmation via SMS or dashboard notification may be added later.

Table 3.41: Use Case – Reschedule Reminder (Conversation)

Use Case ID	UC-3.3
Use Case Name	Detect Wrong Number
Actor(s)	Customer, System
Stakeholders & Interests	Service providers must avoid repeated calls to incorrect numbers to prevent misuse and regulatory violations.
Trigger	Customer states that the system has reached a wrong number.
Preconditions	An active call session is in progress.
Success Postconditions	Number is flagged as invalid in the database and no further calls are scheduled.
Failure Postconditions	Intent is misclassified and the number is not updated correctly.
Priority	Medium
Complexity	Low

Frequency of Use	Low
Functional Requirement(s) Reference	FR-1.10.4, FR-1.7.1
Main Flow	1. System detects a wrong-number intent from the customer's response. 2. System marks the number as invalid in the database. 3. System terminates the call with an appropriate message.
Alternative Flows	1. If intent is uncertain, system asks for clarification before updating records.
Exceptions	1. Database update failure.
Special Requirements	Number must be added to the Do-Not-Call list to prevent future scheduling.
Assumptions	Customer's response is truthful.
Notes and Issues	System may optionally request the correct contact number for the account holder.

Table 3.42: Use Case – Detect Wrong Number

Use Case ID	UC-3.4
Use Case Name	Detect Sentiment and Adapt Response
Actor(s)	Customer, System
Stakeholders & Interests	Service providers require the AI agent to handle confused or frustrated customers appropriately without escalating negative interactions unnecessarily.
Trigger	Customer response signals confusion, frustration, repeated silence, or unresponsiveness.
Preconditions	An active call session is in progress with transcribed user input available.
Success Postconditions	System detects the sentiment and adapts its response tone or conversation path accordingly.
Failure Postconditions	Sentiment undetected; system continues with the default response path.
Priority	Medium
Complexity	Moderate
Frequency of Use	Moderate
Functional Requirement(s) Reference	FR-1.4.4, FR-1.3.2
Main Flow	1. System analyses transcribed input for sentiment indicators (tone, keywords, hesitation). 2. System classifies sentiment (e.g., confused, unresponsive, cooperative, frustrated). 3. System adapts response phrasing or triggers escalation based on the classification.
Alternative Flows	1. Positive or neutral sentiment detected; conversation proceeds on the normal flow.

Exceptions	1. Sentiment model unavailable; system falls back to default conversation path.
Special Requirements	Must support basic sentiment detection for English and Urdu inputs.
Assumptions	Customer speech is captured with sufficient quality for sentiment analysis.
Notes and Issues	Advanced emotion-based tone modulation in TTS may be added in future versions.

Table 3.43: Use Case – Detect Sentiment and Adapt Response

2.1.4 Module 4 – Admin Dashboard Module

Use Case ID	UC-4.1
Use Case Name	Upload Customer Data via CSV
Actor(s)	Admin, System
Stakeholders & Interests	Administrators need bulk upload functionality to efficiently manage large volumes of customer records for reminder campaigns.
Trigger	Admin uploads a CSV file containing customer data via the admin dashboard.
Preconditions	Admin is authenticated and has access to the data management section.
Success Postconditions	Customer records are validated and stored correctly in the database.
Failure Postconditions	File is rejected due to format errors; admin is notified with error details.
Priority	High
Complexity	Moderate
Frequency of Use	Moderate
Functional Requirement(s) Reference	FR-1.1.2, FR-1.7.4
Main Flow	1. Admin uploads a CSV file from the dashboard. 2. System validates the file structure and column formatting. 3. Valid records are stored in the database; invalid rows are flagged and reported.
Alternative Flows	1. Invalid rows are listed for admin review with specific error descriptions.
Exceptions	1. File size exceeds the configured upload limit.
Special Requirements	Must support the standard CSV template defined for the system.
Assumptions	Admin uses the correct CSV template provided by the system.
Notes and Issues	Excel (.xlsx) file support may be added in a later version.

Table 3.44: Use Case – Upload Customer Data via CSV

Use Case ID	UC-4.2
-------------	--------

Use Case Name	View Call Logs
Actor(s)	Admin
Stakeholders & Interests	Administrators require visibility into call outcomes, transcripts, and system activity for monitoring and auditing.
Trigger	Admin opens the call logs section of the dashboard.
Preconditions	Admin is logged in with appropriate access privileges.
Success Postconditions	Call logs are displayed with filtering and search capabilities.
Failure Postconditions	Logs fail to load due to a database error.
Priority	High
Complexity	Low
Frequency of Use	High
Functional Requirement(s) Reference	FR-1.7.1, FR-1.7.3, FR-1.7.5
Main Flow	1. System retrieves call records from the database. 2. System displays logs with search and filter options (by date, status, outcome). 3. Admin can expand any log entry to view the full transcript.
Alternative Flows	1. No records match the filter criteria; system displays "No records found."
Exceptions	1. Database connection error prevents log retrieval.
Special Requirements	Must support CSV export of filtered call logs. Pagination required for large datasets.
Assumptions	Call records are correctly stored and linked to customer records.
Notes and Issues	Full-text transcript search may be added in a future release.

Table 3.45: Use Case – View Call Logs

Use Case ID	UC-4.3
Use Case Name	View Analytics Dashboard
Actor(s)	Admin
Stakeholders & Interests	Service providers and administrators need insights into call success rates, escalation statistics, and overall system performance.
Trigger	Admin opens the analytics section of the dashboard.
Preconditions	Sufficient call data is available in the database for meaningful metrics.
Success Postconditions	Analytics charts and performance metrics are displayed to the admin.
Failure Postconditions	Data is unavailable or incomplete; dashboard shows default or partial metrics.
Priority	Medium

Complexity	Moderate
Frequency of Use	Moderate
Functional Requirement(s) Reference	FR-1.11.1, FR-1.11.3, FR-1.11.4
Main Flow	1. System retrieves aggregated analytics data from the database. 2. System generates charts for call success rate, escalation rate, intent accuracy, and duration. 3. Admin views and interacts with the displayed metrics.
Alternative Flows	1. Partial data is displayed if the full dataset is not yet available.
Exceptions	1. Visualisation rendering error.
Special Requirements	Must display call success rate and escalation statistics. Supports CSV export of analytics.
Assumptions	Analytics engine and database are operational.
Notes and Issues	Advanced predictive analytics may be added in future versions.

Table 3.46: Use Case – View Analytics Dashboard

Use Case ID	UC-4.4
Use Case Name	Manage User Access and Roles
Actor(s)	Admin, System
Stakeholders & Interests	Organisations require controlled access to the dashboard to prevent unauthorised viewing or modification of call records and system settings.
Trigger	Admin creates, modifies, or revokes a user account within the dashboard.
Preconditions	Admin is authenticated with sufficient administrative privileges.
Success Postconditions	User account is created or updated with the correct role and access permissions.
Failure Postconditions	Account change fails and an error is logged.
Priority	Medium
Complexity	Low
Frequency of Use	Low
Functional Requirement(s) Reference	SEC-3, SEC-5
Main Flow	1. Admin navigates to the user management section. 2. Admin creates or modifies a user account and assigns a role. 3. System applies role-based access permissions via Supabase authentication.
Alternative Flows	1. If the email already exists, admin is notified with the option to modify the existing account.
Exceptions	1. Supabase authentication service is unavailable.

Special Requirements	Role-Based Access Control (RBAC) must be enforced via Supabase.
Assumptions	Supabase authentication is configured and operational.
Notes and Issues	Single sign-on (SSO) integration may be added in future versions.

Table 3.47: Use Case – Manage User Access and Roles

Use Case ID	UC-4.5
Use Case Name	Schedule and Manage Reminder Campaigns
Actor(s)	Admin, System
Stakeholders & Interests	Administrators need the ability to create, configure, and manage outbound call campaigns targeting specific customer groups.
Trigger	Admin creates or modifies a reminder campaign from the dashboard.
Preconditions	Admin is authenticated and customer data has been uploaded.
Success Postconditions	Campaign is scheduled with configured parameters (time window, retry count, language).
Failure Postconditions	Campaign creation fails; error is displayed and no calls are scheduled.
Priority	High
Complexity	Moderate
Frequency of Use	Moderate
Functional Requirement(s) Reference	FR-1.10.1, FR-1.1.1
Main Flow	1. Admin configures campaign parameters including target list, time window, retry count, and language. 2. System validates the configuration and schedules the campaign. 3. System begins initiating calls at the scheduled time.
Alternative Flows	1. Admin pauses or cancels an active campaign from the dashboard.
Exceptions	1. Invalid configuration parameters prevent campaign creation.
Special Requirements	Campaign must respect Do-Not-Call list and permitted calling hours.
Assumptions	Target customer list has been validated and uploaded in advance.
Notes and Issues	Advanced campaign targeting and segmentation logic may be added later.

Table 3.48: Use Case – Schedule and Manage Reminder Campaigns

2.2 Functional Requirements

2.2.1 AI Voice Agent Module

Table 3.1: Functional Requirements – Initiate Outbound Reminder Call

ID	Requirement	Priority	Dependency
FR-1.1.1	The system shall initiate outbound reminder calls using the Twilio telephony API.	High	None
FR-1.1.2	The system shall retrieve customer contact details and preferences from the database before initiating a call.	High	FR-1.1.1
FR-1.1.3	The system shall play a personalised reminder message at the beginning of each call.	High	FR-1.1.1
FR-1.1.4	The system shall retry the call up to a predefined limit if the call is not answered.	Medium	FR-1.1.1
FR-1.1.5	The system shall terminate the call automatically if no response is detected within a defined timeout period.	Medium	FR-1.1.3

Table 3.2: Functional Requirements – Retry Failed Call

ID	Requirement	Priority	Dependency
FR-1.2.1	The system shall capture user voice input during an active call session.	High	FR-1.1.3
FR-1.2.2	The system shall convert captured speech into text using the Whisper STT module.	High	FR-1.2.1
FR-1.2.3	The system shall detect user intent from the transcribed text.	High	FR-1.2.2
FR-1.2.4	The system shall generate a context-aware response based on detected intent.	High	FR-1.2.3
FR-1.2.5	The system shall respond to the user using ElevenLabs TTS-generated speech.	High	FR-1.2.4

Table 3.3: Functional Requirements – Escalate Call to Human Agent

ID	Requirement	Priority	Dependency
FR-1.3.1	The system shall detect ambiguous or unrecognised user responses.	High	FR-1.2.3
FR-1.3.2	The system shall escalate the call to a human agent if intent confidence falls below a predefined threshold.	High	FR-1.3.1
FR-1.3.3	The system shall transfer the active call session to an available human agent.	High	FR-1.3.2
FR-1.3.4	The system shall provide the human agent with conversation history and detected intents at the time of transfer.	High	FR-1.3.3
FR-1.3.5	The system shall log all escalation events for monitoring and reporting purposes.	Medium	FR-1.3.2

Table 3.4: Functional Requirements – Manage Conversation Flow

ID	Requirement	Priority	Dependency
FR-1.4.1	The system shall maintain the state of the conversation throughout the call session.	High	None
FR-1.4.2	The system shall store previous user responses for contextual continuity within the session.	High	FR-1.4.1
FR-1.4.3	The system shall support predefined conversation paths for payment confirmation, rescheduling, and wrong-number detection.	High	FR-1.4.1
FR-1.4.4	The system shall implement retry logic for unclear or incomplete user responses.	Medium	FR-1.4.2
FR-1.4.5	The system shall terminate the conversation politely after a task is completed or maximum retries are reached.	Medium	FR-1.4.3

Table 3.5: Functional Requirements – Process Payment Confirmation

ID	Requirement	Priority	Dependency
FR-1.5.1	The system shall identify payment confirmation responses during the call.	High	FR-1.2.3
FR-1.5.2	The system shall verify the payment status with the backend database.	High	FR-1.5.1
FR-1.5.3	The system shall update the reminder status upon payment confirmation.	High	FR-1.5.2
FR-1.5.4	The system shall verbally confirm the successful payment acknowledgment to the user.	Medium	FR-1.5.3
FR-1.5.5	The system shall log confirmed payments for reporting and auditing.	Medium	FR-1.5.3
FR-1.5.6	The system shall operate with synthetic data only and shall not process real financial transactions in the prototype.	High	None

Table 3.6: Functional Requirements – Reschedule Reminder

ID	Requirement	Priority	Dependency
FR-1.6.1	The system shall detect user requests to reschedule reminders.	High	FR-1.2.3
FR-1.6.2	The system shall extract the new requested date and time from user input.	High	FR-1.6.1
FR-1.6.3	The system shall validate the requested date and time for correctness and time conflicts.	High	FR-1.6.2
FR-1.6.4	The system shall update the reminder schedule in the database with the new date/time.	High	FR-1.6.3
FR-1.6.5	The system shall confirm the updated schedule verbally before ending the call.	Medium	FR-1.6.4

Table 3.7: Functional Requirements – Log and Store Call Records

ID	Requirement	Priority	Dependency
FR-1.7.1	The system shall record metadata for every outbound call including duration, timestamp, and outcome.	High	FR-1.1.1
FR-1.7.2	The system shall store call duration, timestamp, and call status for every session.	High	FR-1.7.1
FR-1.7.3	The system shall store full call transcripts generated during the session.	High	FR-1.2.2
FR-1.7.4	The system shall associate call logs with the corresponding customer record.	High	FR-1.7.1
FR-1.7.5	The system shall securely archive call logs in Supabase for future auditing and reporting.	Medium	FR-1.7.2

Table 3.8: Functional Requirements – Detect and Handle Call Failures

ID	Requirement	Priority	Dependency
FR-1.8.1	The system shall detect call failures including busy signal, unreachable, and network errors.	High	FR-1.1.1
FR-1.8.2	The system shall log the reason for each call failure linked to the customer record.	High	FR-1.8.1
FR-1.8.3	The system shall automatically retry failed calls based on predefined retry policies.	Medium	FR-1.8.1
FR-1.8.4	The system shall notify administrators if repeated call failures for a campaign exceed a threshold.	Medium	FR-1.8.3
FR-1.8.5	The system shall mark reminders as permanently failed after maximum retry attempts are exhausted.	High	FR-1.8.3

Table 3.9: Functional Requirements – Personalise Voice Interaction

ID	Requirement	Priority	Dependency
FR-1.9.1	The system shall personalise reminder messages using customer-specific information (name, amount, due date).	High	FR-1.1.2
FR-1.9.2	The system shall support multiple voice profiles for TTS output selection.	Medium	FR-1.2.5
FR-1.9.3	The system shall adjust speech tone and speed based on predefined campaign configurations.	Medium	FR-1.9.2
FR-1.9.4	The system shall support multilingual reminder messages with initial support for English and Urdu.	High	FR-1.2.5
FR-1.9.5	The system shall detect the preferred language from the user profile settings or from spoken input.	Medium	FR-1.9.4

Table 3.10: Functional Requirements – Enforce Call Compliance Rules

ID	Requirement	Priority	Dependency
FR-1.10.1	The system shall ensure calls are placed only within configured permitted calling time windows.	High	FR-1.1.1
FR-1.10.2	The system shall check customer opt-out status before initiating any call.	High	FR-1.1.2
FR-1.10.3	The system shall immediately terminate calls upon a live opt-out request from the customer.	High	FR-1.2.3
FR-1.10.4	The system shall maintain a Do-Not-Call list and prevent scheduling calls to listed numbers.	High	FR-1.10.2
FR-1.10.5	The system shall log all compliance-related events (opt-outs, blocked calls) for auditing.	Medium	FR-1.10.1

Table 3.11: Functional Requirements – Monitor AI Performance Metrics

ID	Requirement	Priority	Dependency
FR-1.11.1	The system shall calculate call success rates for completed reminder campaigns.	High	FR-1.7.2
FR-1.11.2	The system shall measure intent recognition accuracy across call sessions.	High	FR-1.2.3
FR-1.11.3	The system shall track average call duration metrics per campaign.	Medium	FR-1.7.2
FR-1.11.4	The system shall generate performance summaries accessible to administrators via the dashboard.	Medium	FR-1.11.1
FR-1.11.5	The system shall log AI confidence scores for analysis and model improvement.	Medium	FR-1.2.3

2.2.2 Speech Processing Module

Table 3.12: Functional Requirements – Convert Speech to Text

ID	Requirement	Priority	Dependency
FR-2.1.1	The system shall capture real-time audio input from the active Twilio call stream.	High	None
FR-2.1.2	The system shall process captured audio using the Whisper STT engine for transcription.	High	FR-2.1.1
FR-2.1.3	The system shall return transcription results with a confidence score to the conversation module.	High	FR-2.1.2
FR-2.1.4	The system shall request clarification from the user if the transcription confidence is below the configured threshold.	Medium	FR-2.1.3
FR-2.1.5	The system shall support multi-language transcription with initial support for English and Urdu.	High	FR-2.1.2

Table 3.13: Functional Requirements – Convert Text to Speech

ID	Requirement	Priority	Dependency
FR-2.2.1	The system shall receive text responses from the conversation module and generate speech audio using ElevenLabs.	High	None
FR-2.2.2	The system shall stream generated audio directly to the customer via the active call session.	High	FR-2.2.1
FR-2.2.3	The system shall support configurable voice profiles for different languages or campaigns.	Medium	FR-2.2.1
FR-2.2.4	The system shall use a fallback voice if the primary TTS voice is unavailable.	Medium	FR-2.2.2
FR-2.2.5	The system shall minimise TTS response latency to maintain a natural conversation pace.	High	FR-2.2.2

Table 3.14: Functional Requirements – Detect Language

ID	Requirement	Priority	Dependency
FR-2.3.1	The system shall detect the spoken language from the customer's initial utterance using Whisper language identification.	High	FR-2.1.2
FR-2.3.2	The system shall select the appropriate STT model and TTS voice based on the detected language.	High	FR-2.3.1
FR-2.3.3	The system shall default to English if language detection fails or returns low confidence.	Medium	FR-2.3.1
FR-2.3.4	The system shall ask the user to confirm their preferred language if detection confidence is below threshold.	Medium	FR-2.3.3
FR-2.3.5	The system shall support English and Urdu in the prototype with the ability to extend to further languages.	High	FR-2.3.2

Table 3.15: Functional Requirements – Handle Noise and Improve Recognition Accuracy

ID	Requirement	Priority	Dependency
FR-2.4.1	The system shall apply basic audio pre-processing to reduce background noise before STT transcription.	Medium	FR-2.1.1
FR-2.4.2	The system shall return a confidence score for each transcription to enable threshold-based filtering.	High	FR-2.1.2
FR-2.4.3	The system shall request the user to repeat their response if audio quality is below the acceptable threshold.	Medium	FR-2.4.2
FR-2.4.4	The system shall log instances of low-confidence transcriptions for model improvement analysis.	Low	FR-2.4.2

2.2.3 Conversation Management Module

Table 3.16: Functional Requirements – Confirm Payment

ID	Requirement	Priority	Dependency
FR-3.1.1	The system shall classify the user response as a payment confirmation intent using the intent detection model.	High	FR-1.2.3
FR-3.1.2	The system shall require a minimum confidence threshold before updating the payment status.	High	FR-3.1.1
FR-3.1.3	The system shall update the reminder and payment status in the database upon confirmed intent.	High	FR-3.1.2
FR-3.1.4	The system shall verbally acknowledge the confirmation to the customer before ending the call.	Medium	FR-3.1.3
FR-3.1.5	The system shall log the confirmed payment event with a timestamp for auditing.	Medium	FR-3.1.3

Table 3.17: Functional Requirements – Reschedule Reminder (Conversation)

ID	Requirement	Priority	Dependency
FR-3.2.1	The system shall detect a rescheduling intent from the user's spoken response.	High	FR-1.2.3
FR-3.2.2	The system shall prompt the user to provide a preferred date and time for the follow-up call.	High	FR-3.2.1
FR-3.2.3	The system shall extract and validate the date/time from the user's response.	High	FR-3.2.2
FR-3.2.4	The system shall store the updated reminder schedule in the database.	High	FR-3.2.3
FR-3.2.5	The system shall confirm the new schedule verbally to the customer before ending the call.	Medium	FR-3.2.4

Table 3.18: Functional Requirements – Detect Wrong Number

ID	Requirement	Priority	Dependency
FR-3.3.1	The system shall classify the user response as a wrong-number intent using the intent detection model.	High	FR-1.2.3
FR-3.3.2	The system shall flag the phone number as invalid in the database upon confirmed wrong-number detection.	High	FR-3.3.1
FR-3.3.3	The system shall add the number to the Do-Not-Call list to prevent future scheduling.	High	FR-3.3.2
FR-3.3.4	The system shall terminate the call with an appropriate apology message.	Medium	FR-3.3.2

Table 3.19: Functional Requirements – Detect Sentiment and Adapt Response

ID	Requirement	Priority	Dependency
FR-3.4.1	The system shall analyse transcribed user input for sentiment indicators such as confusion, frustration, or silence.	Medium	FR-1.2.3

ID	Requirement	Priority	Dependency
FR-3.4.2	The system shall classify the detected sentiment into predefined categories (cooperative, confused, hostile).	Medium	FR-3.4.1
FR-3.4.3	The system shall adapt response phrasing or simplify language when a confused or unresponsive sentiment is detected.	Medium	FR-3.4.2
FR-3.4.4	The system shall trigger escalation to a human agent when a hostile or persistently unresponsive sentiment is detected.	High	FR-3.4.2
FR-3.4.5	The system shall log sentiment classifications for model improvement and reporting.	Low	FR-3.4.2

2.2.4 Admin Dashboard Module

Table 3.20: Functional Requirements – Upload Customer Data via CSV

ID	Requirement	Priority	Dependency
FR-4.1.1	The system shall allow administrators to upload customer records via a CSV file from the dashboard.	High	None
FR-4.1.2	The system shall validate the CSV structure including required columns and data formats.	High	FR-4.1.1
FR-4.1.3	The system shall store validated customer records in the Supabase database.	High	FR-4.1.2
FR-4.1.4	The system shall report invalid rows with specific error messages to the admin.	Medium	FR-4.1.2
FR-4.1.5	The system shall reject files that exceed the configured maximum upload size.	Medium	FR-4.1.1

Table 3.21: Functional Requirements – View Call Logs

ID	Requirement	Priority	Dependency
FR-4.2.1	The system shall retrieve and display call records from the database in the admin dashboard.	High	FR-1.7.1
FR-4.2.2	The system shall provide filter and search options (by date, outcome, and customer).	High	FR-4.2.1
FR-4.2.3	The system shall allow admins to expand any call record to view its full transcript.	High	FR-1.7.3
FR-4.2.4	The system shall support CSV export of filtered call log results.	Medium	FR-4.2.2
FR-4.2.5	The system shall implement pagination for call log display when records exceed a defined limit.	Medium	FR-4.2.1

Table 3.22: Functional Requirements – View Analytics Dashboard

ID	Requirement	Priority	Dependency
FR-4.3.1	The system shall display key performance metrics including total calls, success rate, and escalation count.	High	FR-1.11.1
FR-4.3.2	The system shall generate visualisation charts for call outcome trends, intent accuracy, and campaign performance.	High	FR-4.3.1
FR-4.3.3	The system shall allow admins to apply filters by date range, campaign, or outcome type.	Medium	FR-4.3.1
FR-4.3.4	The system shall export analytics data to CSV format.	Medium	FR-4.3.3
FR-4.3.5	The system shall refresh analytics data dynamically without requiring a full page reload.	Medium	FR-4.3.2

Table 3.23: Functional Requirements – Manage User Access and Roles

ID	Requirement	Priority	Dependency
FR-4.4.1	The system shall allow admins to create and manage dashboard user accounts via Supabase.	High	None
FR-4.4.2	The system shall support role assignment (admin, viewer) with appropriate access restrictions.	High	FR-4.4.1
FR-4.4.3	The system shall restrict access to sensitive features based on the assigned role.	High	FR-4.4.2
FR-4.4.4	The system shall record an audit log of all role and permission changes.	Medium	FR-4.4.2

Table 3.24: Functional Requirements – Schedule and Manage Reminder Campaigns

ID	Requirement	Priority	Dependency
FR-4.5.1	The system shall allow admins to create new reminder campaigns with configurable parameters.	High	FR-4.1.3
FR-4.5.2	The system shall validate campaign configuration including calling hours, retry count, and language selection.	High	FR-4.5.1
FR-4.5.3	The system shall schedule and initiate outbound calls for a campaign at the configured time.	High	FR-4.5.2
FR-4.5.4	The system shall allow admins to pause, resume, or cancel an active campaign.	Medium	FR-4.5.3
FR-4.5.5	The system shall display campaign status and progress statistics in the dashboard.	Medium	FR-4.5.3

2.3 Non-Functional Requirements

This section specifies the quality attributes that define how the Nora system performs, behaves, and maintains its integrity under various conditions. These requirements ensure that the system is reliable, usable, performant, and secure throughout its lifecycle. All non-functional requirements are designed to be measurable, testable, and verifiable.

2.3.1 System Reliability

Reliability ensures the system consistently performs automated reminder calls without unexpected failures or data loss.

- **REL-1:** The system shall maintain a minimum uptime of 98% during operational hours.
- **REL-2:** The system shall automatically retry failed outbound calls up to three times before marking the call as failed.
- **REL-3:** The system shall automatically retry failed Speech-to-Text or Text-to-Speech API requests.
- **REL-4:** All scheduled reminder calls shall remain persistent in the database even after system restart or unexpected server failure.
- **REL-5:** Call interaction data including transcripts, timestamps, and detected intent shall be saved immediately after each call session.
- **REL-6:** The system shall record and log all system errors for debugging and monitoring purposes.

2.3.2 User Experience and Usability

Usability requirements ensure that administrators can easily manage reminder campaigns and monitor system performance using the dashboard.

- **USE-1:** The admin dashboard shall allow administrators to schedule reminder calls within five minutes of first use.
- **USE-2:** The dashboard shall provide visual indicators for call states including Completed, Failed, Escalated, and Scheduled.
- **USE-3:** The system shall provide clear voice prompts during customer interactions to guide users through the conversation.
- **USE-4:** The admin dashboard shall load within 3 seconds on a standard broadband connection.
- **USE-5:** The dashboard interface shall follow a responsive design suitable for desktop and tablet devices.
- **USE-6:** The system shall display call transcripts and logs in a readable and structured format for administrators.

2.3.3 Performance

Performance requirements define system responsiveness and AI processing speed during real-time calls.

- **PER-1:** The system shall initiate scheduled outbound reminder calls within 2 seconds of their scheduled time.
- **PER-2:** The system shall support at least 150 concurrent active calls without service degradation.
- **PER-3:** Speech-to-Text processing shall convert user speech to text within 2 seconds after speech completion.
- **PER-4:** Text-to-Speech responses shall be generated and played within 1 second of response generation.
- **PER-5:** AI intent detection shall complete within 1 second after transcription is received.
- **PER-6:** Database read and write operations shall respond within 300 milliseconds under normal system load.
- **PER-7:** Call logs and analytics queries shall return results within 3 seconds for datasets containing up to 50,000 records.

2.3.4 Data Security and Privacy

Security requirements ensure protection of customer data, system access, and communication channels.

- **SEC-1:** All communication between the client, backend server, and telephony APIs shall use HTTPS with TLS encryption.
- **SEC-2:** Administrator authentication shall be implemented using secure Supabase authentication mechanisms.
- **SEC-3:** The system shall implement Role-Based Access Control (RBAC) for administrative users.
- **SEC-4:** Sensitive customer data including phone numbers and transcripts shall be encrypted during storage and transmission.
- **SEC-5:** All administrator login attempts shall be logged for auditing purposes.
- **SEC-6:** API keys and service credentials shall be stored securely using environment variables.
- **SEC-7:** The system shall enforce request rate limiting to protect against denial-of-service attacks.

2.3.5 System Maintainability

Maintainability ensures that the system can be easily updated, extended, and debugged during future development.

- **MAI-1:** The system shall follow a modular architecture separating telephony services, AI processing, speech processing, database management, and dashboard components.
- **MAI-2:** All source code shall follow standardized coding and formatting guidelines.
- **MAI-3:** The system shall include unit and integration tests for critical modules.
- **MAI-4:** System configuration parameters shall be externalized using environment configuration files.
- **MAI-5:** Error logs and monitoring data shall be accessible to developers for troubleshooting and maintenance.

2.3.6 Scalability

Scalability requirements ensure that the system can support increasing numbers of customers and reminder calls.

- **SCA-1:** The system architecture shall support horizontal scaling using cloud infrastructure.
- **SCA-2:** The system shall support storage and retrieval of at least 100,000 call records.
- **SCA-3:** The system shall allow integration of additional languages or voice models without major architectural modifications.
- **SCA-4:** The backend API shall support scaling across multiple server instances.
- **SCA-5:** Analytics queries shall use optimized database indexing to maintain performance with increasing data size.

2.3.7 Deployment and Portability

Portability ensures the system can operate across different environments and platforms.

- **POR-1:** The system shall be deployable on major cloud platforms such as AWS or Google Cloud.
- **POR-2:** The backend services shall support containerized deployment using Docker.
- **POR-3:** The admin dashboard shall function on major web browsers including Chrome, Firefox, Edge, and Safari.
- **POR-4:** The system APIs shall follow RESTful architecture to support integration with external systems.

2.3.8 System Availability

Availability defines the operational accessibility of the system during its lifecycle.

- **AVA-1:** The system shall maintain a minimum availability of 99% in production environments.
- **AVA-2:** Scheduled maintenance downtime shall not exceed 2 hours per month.
- **AVA-3:** A system maintenance notification page shall be displayed during planned downtime.
- **AVA-4:** Load balancing mechanisms shall be implemented to maintain service continuity during high call volumes.

Table 2.78: Summary of Non-Functional Requirements for Nora – Conversational Voice Agent

ID	Category	Description	Metric / Target
REL-1	Reliability	System uptime during operational hours	98% minimum
REL-2	Reliability	Automatic retry limit for failed calls	Up to 3 retries
REL-5	Reliability	Call data saved immediately after session	Per call
USE-1	Usability	Campaign scheduling time for new users	Within 5 minutes
USE-4	Usability	Dashboard load time	3 seconds
PER-1	Performance	Call initiation delay from scheduled time	Within 2 seconds
PER-2	Performance	Concurrent active calls supported	150 calls
PER-3	Performance	STT processing time after speech	Within 2 seconds
PER-4	Performance	TTS generation and playback latency	Within 1 second
PER-5	Performance	Intent detection latency after transcription	Within 1 second
PER-6	Performance	Database read/write latency	300 ms
SEC-1	Security	Secure communication protocol	HTTPS (TLS 1.3)
SEC-3	Security	Access control model	RBAC via Supabase
SEC-4	Security	Data encryption standard	Encrypted in transit and at rest
MAI-3	Maintainability	Test coverage for critical modules	Unit and integration tests
SCA-1	Scalability	Horizontal scaling capability	Cloud-ready (AWS/GCP)
SCA-2	Scalability	Maximum call records supported	100,000+
POR-1	Portability	Cloud platform compatibility	AWS, GCP
POR-2	Portability	Containerized deployment	Docker
AVA-1	Availability	Platform uptime target	99%
AVA-2	Availability	Maximum monthly maintenance window	2 hours

System Overview [AS PER FYP1 MID REPORT]

This chapter provides an overview of the design, structure, and working of the Nora – Conversational Voice Agent system. It outlines the overall architecture, the major subsystems, and how they interact to form an integrated platform for automated payment and service reminders. The objective of this system is to automate outbound reminder calls for service providers, process customer spoken responses in real time, detect intent, and log all interactions for reporting and analytics.

3.1 System Description

Nora is a modular, full-stack conversational voice agent that enables service providers (ISPs, gyms, utility companies, subscription services) to automate payment and service reminders through outbound phone calls. The system initiates two-way calls, understands customer responses using speech recognition, detects intent, generates context-aware replies, and logs every interaction in a web-based admin dashboard.

The system is built with a separation of concerns across three primary layers:

- **Frontend Layer (Client)** – Responsible for the admin dashboard interface. Built with Next.js, React, and Tailwind CSS. Displays call logs, transcripts, detected intents, success/failure rates, follow-up schedules, escalation statistics, and analytics. Communicates with the backend via HTTPS REST API calls.
- **Backend Layer (Server)** – Handles all business logic, voice agent orchestration, speech processing, intent detection, conversation management, and compliance enforcement. Implemented using FastAPI (Python) with asynchronous support for real-time call processing.
- **Data Layer (Database & Storage)** – Manages structured data including call logs, transcripts, user accounts, and conversation states. Uses Supabase (PostgreSQL) for persistent storage and authentication, with Twilio managing the telephony layer.

The architecture follows a modular and loosely coupled design, ensuring that updates to one component (e.g., the intent detection model or TTS engine) do not affect the stability of others. Each module maps directly to a major system function: AI Voice Agent, Speech Processing, Conversation Management, and Admin Dashboard.

3.2 Architectural Design

The architectural design of Nora follows a multi-tiered client-server pattern with an MVC (Model-View-Controller) orientation within the web layer. It separates data handling, business logic, and presentation components to improve scalability, maintainability, and modularity.

3.2.1 Module Relationships

Each subsystem is responsible for a defined function but communicates through standardised APIs:

- The AI Voice Agent Module initiates outbound calls via the Twilio telephony API, delivers pre-defined reminder messages, manages call flow (confirmations, retries, termination), escalates to human agents when unable to resolve a request, and logs call-related metadata.
- The Speech Processing Module converts customer voice input to text using Whisper STT, generates natural-sounding audio responses using ElevenLabs TTS, performs language detection, and applies basic noise handling. Supports English and Urdu.

- The Conversation Management Module maintains conversation state across multiple turns, tracks previous user responses, handles conversation paths (payment confirmation, reminder rescheduling, escalation), incorporates simple sentiment awareness, and enforces retry logic and escalation rules.
- The Admin Dashboard Module provides a web-based interface displaying call logs, transcripts, detected intents, success/failure rates, follow-up schedules, escalation statistics, and analytics. Supports CSV export and user access management.
- The Compliance Engine enforces call frequency limits, Do-Not-Call regulations, and call termination rules.
- The Deployment & Infrastructure Layer uses Docker for containerised deployment, Cloud VMs (AWS/GCP) for hosting, and CI/CD pipelines for automated testing and deployment.

3.2.2 Subsystem Collaboration

When a call is initiated, the following workflow occurs:

1. The Admin Dashboard triggers an outbound call. The Compliance Engine checks the customer's Do-Not-Call flag and call frequency limit.
2. The AI Voice Agent Module initiates the call via the Twilio API.
3. The pre-defined reminder message is synthesised by ElevenLabs TTS and streamed to the customer via Twilio.
4. The customer's voice response is captured; Whisper STT converts it to text with language detection and noise handling applied.
5. The Conversation Management Module classifies the intent (ConfirmPayment / ScheduleFollowUp / Escalate / Unclear) using the intent detection model via Groq inference.
6. Depending on the detected intent, the system confirms payment, schedules a follow-up, or escalates to a human agent.
7. The call interaction is logged to Supabase and the Admin Dashboard is updated with the outcome, transcript, and analytics.

3.2.3 Design Principles

The overall architecture is based on the following principles:

- Separation of Concerns: Each layer focuses on a single responsibility – presentation, business logic, and data management are cleanly separated.
- Modularity: All four modules (AI Voice Agent, Speech Processing, Conversation Management, Admin Dashboard) are independently maintainable and testable.
- Reliability: Includes retry logic ($\text{retryCount} \leq 3$), escalation fallback to human agents, and Supabase-backed persistent logging to prevent data loss.
- Compliance: Enforces Do-Not-Call regulations, call frequency limits, and call termination rules through the dedicated Compliance Engine.
- Extensibility: Future support for additional languages, advanced sentiment models, real payment gateway integration, and predictive analytics is architecturally supported.

3.2.4 System Context

Nora interacts with several external systems:

- Twilio (Telephony API): For outbound/inbound call management, call routing, and real-time audio streaming in test mode.
- Whisper by OpenAI (STT): For real-time, multi-language speech-to-text conversion with language detection and noise robustness.

- ElevenLabs (TTS): For natural-sounding AI speech synthesis in outbound calls, supporting configurable voice profiles.
- Groq (AI Inference): High-performance inference platform for intent detection and conversation modelling using large language models.
- Supabase: Cloud-hosted PostgreSQL database with built-in authentication, RBAC, and access control for all system data.
- Cloud VMs (AWS/GCP): Scalable hosting for backend, frontend, and all services with Docker containerisation.

3.3 Diagrams to Include

3.3.1 Box and Line Diagram (High-Level Context Diagram)

Purpose: To show all major system modules and how they interact through data and control flow. External entities (Customer, Admin, Service Provider, Human Agent) are shown at the boundary of the system. Internal modules (AI Voice Agent, Speech Processing, Conversation Management, Admin Dashboard, Twilio, Supabase) are shown as labelled boxes connected by directed, labelled arrows.

[Figure 4.1: Box and Line Diagram – refer to Appendix C for the full diagram]

3.3.2 Multi-Tier (Layered/MVC) Architecture Diagram

Purpose: To show logical decomposition and layering across three tiers: Presentation (Next.js + React + Tailwind CSS), Business Logic (FastAPI, Voice Agent Engine, Speech Processing Service, Conversation Management Service, Compliance Engine, Intent Detection Module, CI/CD Pipeline), and Data & Infrastructure (Supabase, Twilio, Whisper, ElevenLabs, Groq, Docker, Cloud VMs).

[Figure 4.2: Architecture Diagram – refer to Appendix C for the full diagram]

3.4 Design Models

This section presents the design models of the Nora – Conversational Voice Agent system, developed following an object-oriented software design approach. Each model provides a visual and structural representation of how different system components interact, process data, and transition between states. These models ensure modularity, maintainability, and alignment with the functional and non-functional requirements of the system.

3.4.1 Activity Diagram

The activity diagrams represent the dynamic workflow of the automated outbound reminder call process. The process is divided into multiple diagrams to clearly illustrate the sequence of activities across different modules including the Admin, AI Voice Agent Module, Speech Processing Module, Conversation Management Module, Customer, and Human Agent (Call Center). These diagrams also illustrate decision points such as compliance validation, customer answer detection, intent classification, retry limits, and escalation conditions.

Key Flow:

- Admin triggers the outbound call from the Admin Dashboard.
- The Compliance Engine checks the customer's Do-Not-Call flag and call frequency count before the call is initiated.

- The AI Voice Agent initiates the call via Twilio. If no answer is received, the system retries up to 3 times before logging the call as Failed.
- On connection, ElevenLabs TTS synthesises and delivers the pre-defined reminder message.
- The customer's voice response is captured; Whisper STT converts it to text with language detection and noise handling.
- If no valid speech is detected, the system re-prompts up to 3 times before escalating to a human agent.
- The Conversation Management Module runs intent detection (ConfirmPayment / ScheduleFollowUp / Escalate / Unclear) with sentiment awareness to adapt responses when the user appears confused or unresponsive.
- Based on the detected intent, the system confirms payment, schedules a follow-up (capturing the new date/time), or escalates to a human agent.
- All call interactions are logged to Supabase and the Admin Dashboard is updated.

[Figures 4.3–4.8: Activity Diagram Parts 1–6 – refer to Appendix D for all six parts]

3.4.2 Class Diagram

The class diagram defines the main system entities, their attributes, methods, and relationships in the object-oriented architecture. It models ten classes: Customer, Call, Transcript, Utterance, ConversationState, DetectedIntent, SpeechProcessor, AdminUser, CallLog, and ComplianceEngine.

Key Relationships:

- A Customer can receive many Calls (1..*).
- Each Call produces one Transcript (1..1), one ConversationState (1..1), one DetectedIntent (1..1), and one CallLog (1..1).
- A Transcript contains many Utterances (1..*).
- ConversationState references DetectedIntent (1..1).
- A Call may be escalated to one HumanAgent (0..1).
- ComplianceEngine checks many Customers (1..*).
- AdminUser views many CallLogs (1..*).

[Figure 4.9: Class Diagram – refer to Appendix D for the full diagram]

3.4.3 Class-Level Sequence Diagram

These class-level sequence diagrams illustrate the sequence of interactions between system classes during an outbound reminder call. The workflow is divided into multiple diagrams to clearly represent the exchange of messages between objects when a call is triggered. The diagrams cover the complete payment-confirmed path and also include alternative fragments for escalation and unclear-intent retry scenarios.

Scenario Example: Outbound Reminder Call – Payment Confirmed

8. AdminDashboard calls VoiceAgentEngine.triggerOutboundCall(customerID).
9. VoiceAgentEngine calls ComplianceEngine.isCallAllowed(customerID) – verifies Do-Not-Call status and call frequency limit.
10. VoiceAgentEngine calls TwilioClient.initiateCall(phoneNumber).

11. Reminder message is synthesised via ElevenLabs TTS and streamed through Twilio.
12. Customer responds; Whisper STT transcribes the audio with language detection and noise handling.
13. ConversationManager calls `DetectedIntent.classifyIntent(text)` via Groq inference, receiving `intent=ConfirmPayment`, `confidence=0.91`.
14. VoiceAgentEngine delivers the confirmation response (TTS), calls `Supabase.saveCallLog()`, then calls `TwilioClient.endCall()`.

[Figures 4.10–4.14: Class-Level Sequence Diagram Parts 1–5 – refer to Appendix D]

3.4.4 State Transition Diagram

The state transition diagram models how a Call session moves through various states during its lifecycle. It focuses on events such as call connection, customer speech detection, intent classification results, retry limits, and escalation triggers.

Example States:

- Idle
- Initiated
- Connected / No Answer
- Awaiting Response
- Processing Intent
- Retrying
- Payment Confirmed / Follow-Up Scheduled / Escalated
- Closed / Failed

[Figure 4.15: State Transition Diagram – refer to Appendix D for the full diagram]

3.5 Data Design

The data design of Nora defines how information from the voice call process, speech recognition, intent detection, and conversation management is structured, stored, and retrieved efficiently. The goal is to ensure secure persistence, fast access, and consistency across all user and administrative operations.

3.5.1 Overview

The system's information domain revolves around five primary data entities: Customer, Call, Transcript, ConversationState, and DetectedIntent. These entities interact through defined relationships to support the core functionalities of the application – including outbound call management, real-time speech processing, intent classification, and admin reporting. Data is stored in Supabase (cloud-hosted PostgreSQL), with authentication and access control managed through Supabase Auth. All data in transit is secured using TLS. Passwords are hashed using bcrypt.

3.5.2 Data Flow and Processing

- When an Admin triggers a call, the customer record is fetched from D1: Customer Records and the Compliance Engine validates eligibility.
- The AI Voice Agent initiates the call via Twilio and the reminder message is synthesised by ElevenLabs TTS.

- The customer's voice response is processed by Whisper STT; language is detected and noise is filtered before the transcribed text is passed to the Conversation Management Module.
- The intent detection model (via Groq) classifies the text; the ConversationState is updated with the new state, turn count, retry count, and sentiment flag.
- Once the call concludes, the full interaction is saved to D2: Call Logs and D3: Transcripts & Intents in Supabase.
- The Admin Dashboard reads from these stores to display analytics, call logs, transcripts, and follow-up schedules.

3.5.3 Major Entities and Attributes

Customer – Stores end-user data and compliance flags

- customerID – Unique identifier
- name, phoneNumber, email
- preferredLanguage – {English, Urdu}
- doNotCall – Boolean flag
- callFrequencyCount – Integer
- createdAt – DateTime

Call – Core entity representing a single outbound call session

- callID, customerID
- startTime, endTime, duration (seconds)
- status – {Initiated, Connected, NoAnswer, Completed, Failed, Escalated}
- retryCount – Integer
- twilioCallSID – String
- outcome – {PaymentConfirmed, FollowUpScheduled, Escalated, Failed, NoAnswer}
- createdAt – DateTime

Transcript – Full text record of the call conversation

- transcriptID, callID
- fullText – String
- language – {English, Urdu}
- createdAt – DateTime

ConversationState – Tracks the live state of each call

- stateID, callID
- currentState – {Idle, Greeting, ReminderDelivered, AwaitingResponse, Retrying, PaymentConfirmed, FollowUpScheduled, Escalated, Closed}
- previousResponses – List<String>
- turnCount, retryCount – Integer
- sentimentFlag – {Neutral, Confused, Unresponsive}
- updatedAt – DateTime

DetectedIntent – Result of intent classification per call

- intentID, callID
- intent – {ConfirmPayment, ScheduleFollowUp, Escalate, Unclear}
- confidence – Float (0.0–1.0)
- transcribedInput – String
- timestamp – DateTime

3.5.4 Data Storage and Security

- **Primary Database:** Supabase (PostgreSQL) for persistent structured data including call logs, transcripts, user accounts, and conversation states.
- **Authentication:** Supabase Auth with role-based access control ({Admin, Viewer}) enforced via JWT.
- **Security:** All data in transit is secured using TLS 1.3. Passwords are hashed using bcrypt. Sensitive data is encrypted at rest.
- **Backups:** Automated daily snapshots. Containerised deployment via Docker on Cloud VMs (AWS/GCP) ensures environment consistency.

3.5.5 Dataflow Diagrams

[Figure 4.16: DATA FLOW DIAGRAM – Level 0 (Context Diagram)]

[Figure 4.17: DATA FLOW DIAGRAM – Level 1 (Process Decomposition)]

[Figure 4.18: Data Flow Diagram – Level 2 (Sub-process Detail)]

[Figures 4.19–4.26: System Level Sequence Diagram Parts 1–8]

3.6 Domain Model

The domain model captures the key conceptual entities of the Nora system and their relationships, independent of implementation details. Core entities include Customer, Call, Transcript, ConversationState, DetectedIntent, ComplianceRecord, AdminUser, and CallLog.

[Figure 4.27: Domain Model Diagram – refer to Appendix B for the full diagram]

Implementation and Testing [UPTO THE CURRENT ITERATION ONLY]

The implementation of Nora – Conversational Voice Agent follows a modular, scalable, and highly interactive architecture that combines real-time telephony, speech-to-text transcription, intent and sentiment classification, text-to-speech synthesis, and a modern web-based admin dashboard. The system is built using FastAPI (Python) for the backend, Next.js with React and Tailwind CSS for the frontend, Twilio for telephony, Whisper for STT, ElevenLabs for TTS, Groq for AI inference, and Supabase for persistent data storage.

To ensure performance and modularity, the application integrates an asynchronous FastAPI backend with a Supabase-backed data layer, a configurable Twilio call orchestration engine, and a real-time admin dashboard with call log viewing and analytics. The implementation is iterative, focusing first on core voice agent features such as outbound call initiation, speech processing, intent detection, and conversation management.

The following sections describe the primary algorithm, external APIs and SDKs integrated, and the unit testing strategy implemented up to the current iteration.

4.1 Algorithm Design

This section outlines the core algorithm that powers the system's call orchestration. The algorithm is presented with a conceptual overview followed by a visual pseudocode representation illustrating the logical flow and implementation approach used in the current iteration.

4.1.1 Outbound Reminder Call Orchestration Algorithm

The primary algorithm driving Nora is the Outbound Reminder Call Orchestration routine. It is responsible for retrieving a scheduled customer record, initiating a call via Twilio, streaming a personalised TTS reminder through ElevenLabs, capturing and transcribing the customer's spoken response via Whisper, classifying the intent and sentiment using the Groq inference API, and branching into one of four outcomes: PaymentConfirmed, FollowUpScheduled, Escalated, or Failed. A configurable retry loop handles unanswered calls and unclear responses before ultimately logging the result to Supabase.

Purpose:

- Retrieve scheduled customer record and validate compliance before initiating the call.
- Initiate outbound call via Twilio and deliver a personalised TTS reminder message.
- Capture customer's spoken response, transcribe via Whisper STT with language detection and noise handling.
- Classify intent and detect sentiment using Groq inference engine.
- Branch to the appropriate outcome (PaymentConfirmed, FollowUpScheduled, Escalated, or Failed) based on intent and confidence.
- Log the complete call interaction to Supabase for reporting and analytics.

Figure 5.1: Algorithm Design – Outbound Reminder Call Orchestration

```
ALGORITHM OutboundReminderCallOrchestration
```

```

INPUT:  customerID, campaignConfig
OUTPUT: callOutcome {PaymentConfirmed | FollowUpScheduled | Escalated | Failed}

1.  customer ← DB.fetchCustomer(customerID)
2.  IF ComplianceEngine.isBlocked(customer) THEN
3.      RETURN Failed (DNC or out-of-hours)
4.  retryCount ← 0
5.  WHILE retryCount ≤ campaignConfig.maxRetries DO
6.      callSID ← Twilio.initiateCall(customer.phoneNumber)
7.      IF call not answered THEN retryCount ← retryCount + 1; continue
8.      message ← buildPersonalisedMessage(customer, campaignConfig)
9.      ElevenLabs.streamTTS(message, callSID)
10.     audio ← Twilio.captureAudio(callSID)
11.     text ← Whisper.transcribe(audio)
12.     intent, confidence ← GroqInference.classifyIntent(text)
13.     sentiment ← SentimentEngine.detect(text)
14.     IF confidence < threshold OR sentiment = Hostile THEN
15.         RETURN Escalated
16.     SWITCH intent DO
17.         ConfirmPayment ⇒ DB.updateStatus(CONFIRMED); RETURN PaymentConfirmed
18.         ScheduleFollowUp ⇒ newDate ← extractDate(text); DB.reschedule(customerID, newDate); RETURN FollowUpScheduled
19.         Unclear ⇒ retryCount ← retryCount + 1
20.     DB.logCall(customerID, callSID, Failed)
21.     RETURN Failed

```

4.2 External APIs and SDKs Used

The implementation of the Nora system relies on several third-party APIs and SDKs that support core functionalities such as telephony, speech-to-text, text-to-speech, AI inference, data persistence, and backend orchestration. These external tools enhance system performance, reduce development time, and ensure the reliability of essential features.

Table 5.1: External APIs and SDKs Used in Nora

API and Version	Description	Purpose in System	Methods / Endpoints Used
Twilio Programmable Voice (2026)	Telephony API for outbound/inbound call management and real-time audio streaming.	Initiating outbound reminder calls and streaming TTS audio to customers.	<code>client.calls.create()</code>
Whisper by OpenAI (v3, 2023)	Multilingual speech-to-text model supporting real-time transcription.	Converting customer voice responses to text during active calls.	<code>openai.audio.transcriptions.create()</code>

API and Version	Description	Purpose in System	Methods / Endpoints Used
ElevenLabs TTS (2026)	Text-to-speech engine generating natural-sounding AI voice responses.	Synthesising personalised reminder messages and conversational replies.	<code>/v1/text-to-speech/{voice_id}</code>
Groq Inference API (2025)	High-performance LLM inference platform for intent and sentiment classification.	Classifying customer intent: ConfirmPayment, Reschedule, Escalate, Unclear.	<code>client.chat.completions.create()</code>
Supabase (v2, 2026)	Cloud-hosted PostgreSQL database with built-in authentication and RBAC.	Storing call logs, transcripts, customer records, and admin user accounts.	<code>supabase.from().insert()</code> / <code>.select()</code>
FastAPI (v0.111, 2026)	Asynchronous Python web framework for building REST API endpoints.	Handling all backend API requests, call orchestration, and business logic.	<code>@app.post()</code> / <code>@app.get()</code>

4.3 Unit Testing

Unit testing was conducted to verify the correctness of individual components of the Nora Conversational Voice Agent system. Each unit test focuses on a single function, module, or workflow to ensure that core features behave as expected under controlled conditions. The testing process included intent classification testing, API response validation, conversation state management testing, and compliance rule enforcement verification.

4.3.1 Intent Classification Tests

The primary unit test suite covers the `ConversationManager.classifyIntent()` function, which is central to the call outcome logic. Tests were executed using the `pytest` framework.

Figure 5.2: Unit Testing – Intent Classification Module

```

Module:      ConversationManager.classifyIntent()
Framework:  pytest

def test_payment_confirmed_intent():
    inputs = [
        "Yes I have already paid",
        "Maine payment kar di hai",
        "Payment done yesterday"
    ]
    for text in inputs:
        result = classify_intent(text)

```

```

        assert result["intent"] == "ConfirmPayment"
        assert result["confidence"] >= 0.80
def test_reschedule_intent():
    result = classify_intent("Call me back on Friday")
    assert result["intent"] == "ScheduleFollowUp"
def test_unclear_intent_triggers_retry():
    result = classify_intent("umm... I don't know")
    assert result["intent"] == "Unclear"
    assert result["confidence"] < 0.60

```

4.3.2 Additional Unit Test Cases

Test Case ID	TC001
Test Objective	Verify system initiates outbound call successfully.
Description	System calls a scheduled customer via Twilio when the reminder time is reached.
Precondition	Customer record exists in DB with a valid phone number and scheduled reminder.
Steps	1. Trigger campaign scheduler. 2. System retrieves customer record. 3. System initiates Twilio call.
Test Data	Customer ID: 101, Phone: +923001234567
Expected Result	Call is initiated; Twilio returns a valid callSID.
Post-condition	Call log is created with status Initiated.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC002
Test Objective	Verify compliance check blocks DNC customers.
Description	System checks Do-Not-Call status before initiating any call.
Precondition	Customer record with doNotCall = True exists in DB.
Steps	1. Trigger call for DNC customer. 2. System checks compliance. 3. Call is blocked.
Test Data	Customer ID: 102, doNotCall: True
Expected Result	Call is not initiated; reason logged as DNC violation.
Post-condition	Call log updated with status Blocked (DNC).
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC003
Test Objective	Verify Whisper STT converts speech to text correctly.

Description	Speech audio from an active Twilio call is transcribed by Whisper.
Precondition	Active call session with audio input stream available.
Steps	1. Stream audio from Twilio call. 2. Pass to Whisper STT. 3. Return transcription.
Test Data	Audio: 'Yes I have already paid my bill'
Expected Result	Transcription: 'Yes I have already paid my bill' with confidence ≥ 0.85 .
Post-condition	Transcribed text passed to intent module.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC004
Test Objective	Verify ElevenLabs TTS generates and streams audio correctly.
Description	System receives text response and streams TTS audio to customer.
Precondition	Active Twilio call session available; ElevenLabs API configured.
Steps	1. System generates text response. 2. Call ElevenLabs TTS API. 3. Stream audio to Twilio.
Test Data	Text: 'Thank you for confirming your payment.'
Expected Result	Natural-sounding audio streamed to customer without delay.
Post-condition	Audio played successfully; call remains active.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC005
Test Objective	Verify Groq intent detection correctly classifies payment confirmation.
Description	Transcribed user text is classified as ConfirmPayment intent.
Precondition	Transcribed text available; Groq API accessible.
Steps	1. Pass transcription to Groq inference. 2. Receive intent and confidence.
Test Data	Text: 'Yes, payment kar diya hai'
Expected Result	intent=ConfirmPayment, confidence ≥ 0.80
Post-condition	Conversation path branches to payment confirmation flow.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC006
Test Objective	Verify rescheduling intent is detected and new date is extracted.

Description	System detects rescheduling request and extracts new date/time.
Precondition	Active call session in progress; user speaks rescheduling request.
Steps	1. User says 'Call me back on Friday'. 2. System classifies intent. 3. System extracts date.
Test Data	Text: 'Call me back on Friday afternoon'
Expected Result	intent=ScheduleFollowUp, date=next Friday extracted.
Post-condition	Reminder schedule updated in DB; confirmation played.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC007
Test Objective	Verify call escalation when intent confidence is below threshold.
Description	System escalates to human agent when confidence is too low.
Precondition	Active call session; multiple unclear user responses detected.
Steps	1. User gives unclear response 3 times. 2. System detects low confidence. 3. Escalation triggered.
Test Data	intent=Unclear, confidence=0.42
Expected Result	Call transferred to human agent; conversation history passed.
Post-condition	Escalation event logged in DB and admin dashboard updated.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC008
Test Objective	Verify call retry logic for unanswered calls.
Description	System retries call up to configured maximum if unanswered.
Precondition	Customer does not answer on first attempt; retry policy set to 3.
Steps	1. System initiates call. 2. Call goes unanswered. 3. System retries after interval.
Test Data	retryCount=1, maxRetries=3
Expected Result	Second call attempt initiated after configured retry interval.
Post-condition	retryCount incremented; call log updated per attempt.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC009
Test Objective	Verify call log is created after call completion.

Description	All call metadata, transcript, and outcome are stored in Supabase.
Precondition	Call session ends (successfully or failed).
Steps	1. Call ends. 2. System calls DB.logCall(). 3. Record created in Supabase.
Test Data	callID=789, status=PaymentConfirmed, duration=45s
Expected Result	Call log record created with all fields populated.
Post-condition	Log accessible via admin dashboard.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC010
Test Objective	Verify wrong number detection updates DNC list.
Description	System detects wrong-number intent and flags number as invalid.
Precondition	Active call; customer states 'wrong number'.
Steps	1. User says 'I think you have the wrong number'. 2. System classifies intent. 3. Number flagged and DNC updated.
Test Data	Text: 'You have the wrong number'
Expected Result	intent=WrongNumber detected; number added to DNC list.
Post-condition	Customer record updated; no future calls scheduled.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC011
Test Objective	Verify language detection selects Urdu processing for Urdu speakers.
Description	System detects Urdu language and switches to appropriate STT/TTS configuration.
Precondition	Customer speaks Urdu at the start of the call.
Steps	1. Customer speaks in Urdu. 2. Whisper detects language. 3. System selects Urdu voice profile.
Test Data	Audio: Urdu spoken input
Expected Result	Language detected as Urdu; Urdu TTS voice selected.
Post-condition	Subsequent responses delivered in Urdu.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC012
Test Objective	Verify admin can upload customer data via CSV.

Description	System validates and stores CSV-uploaded customer records.
Precondition	Admin logged into dashboard; valid CSV file prepared.
Steps	1. Admin uploads CSV. 2. System validates structure. 3. Valid records stored in DB.
Test Data	CSV with 100 customer records
Expected Result	All valid records stored; invalid rows reported with errors.
Post-condition	Customer records available for campaign scheduling.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC013
Test Objective	Verify call logs are visible and filterable in admin dashboard.
Description	Admin dashboard displays call records with filter and search capabilities.
Precondition	Multiple call records exist in Supabase.
Steps	1. Admin opens call logs section. 2. Applies date filter. 3. Views filtered results.
Test Data	Filter: Date = today
Expected Result	Filtered call logs displayed with transcript expand option.
Post-condition	Logs viewable and CSV export functional.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC014
Test Objective	Verify analytics dashboard displays correct call metrics.
Description	Dashboard shows total calls, success rate, and escalation statistics.
Precondition	Sufficient call records exist in DB for meaningful aggregation.
Steps	1. Admin opens analytics. 2. System aggregates data. 3. Charts rendered.
Test Data	10 calls: 7 Confirmed, 2 Escalated, 1 Failed
Expected Result	Success rate: 70%, Escalation rate: 20% displayed correctly.
Post-condition	Dashboard refreshes dynamically without page reload.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC015
Test Objective	Verify sentiment detection adapts conversation when user seems confused.

Description	System detects confused sentiment and simplifies next response.
Precondition	Active call; user gives hesitant, unclear response.
Steps	1. User speaks hesitantly. 2. System detects Confused sentiment. 3. System rephrases next prompt.
Test Data	Text: 'Umm... I am not sure... what?'
Expected Result	Sentiment classified as Confused; simplified rephrasing delivered.
Post-condition	Conversation continues with simplified prompt; no unnecessary escalation.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC016
Test Objective	Verify personalised message uses customer name and amount correctly.
Description	System builds personalised reminder message from customer record fields.
Precondition	Customer record includes name, amount, and due date.
Steps	1. System fetches customer record. 2. Build personalised message. 3. Deliver via TTS.
Test Data	name='Ahmed Ali', amount='PKR 5,000', dueDate='May 15 2026'
Expected Result	Message: 'Hello Ahmed Ali, your payment of PKR 5,000 is due on May 15 2026.'
Post-condition	Personalised message delivered in correct language.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC017
Test Objective	Verify compliance rule blocks call outside permitted hours.
Description	System checks calling time window and blocks out-of-hours calls.
Precondition	Customer record exists; current time is outside permitted hours.
Steps	1. Scheduler triggers call. 2. Compliance Engine checks time. 3. Call blocked.
Test Data	Calling hours: 9 AM – 6 PM; current time: 8 PM
Expected Result	Call not initiated; reason logged as out-of-hours.
Post-condition	No call attempt made; admin dashboard reflects blocked status.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC018
Test Objective	Verify DB.reschedule() stores updated reminder schedule correctly.

Description	Database is updated with new reminder date when user requests rescheduling.
Precondition	Rescheduling intent detected; new date/time extracted from user input.
Steps	1. intent=ScheduleFollowUp detected. 2. newDate extracted. 3. DB.reschedule() called.
Test Data	customerID=101, newDate=Friday May 16 2026 3PM
Expected Result	Reminder schedule updated in DB with new date; confirmation played.
Post-condition	Updated schedule visible in admin dashboard.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC019
Test Objective	Verify admin can create and schedule a reminder campaign.
Description	System validates campaign configuration and schedules outbound calls.
Precondition	Admin authenticated; customer data uploaded.
Steps	1. Admin creates campaign with parameters. 2. System validates config. 3. Campaign scheduled.
Test Data	Campaign: Retry=3, Hours=9AM-6PM, Language=English
Expected Result	Campaign created and scheduled; calls initiated at configured time.
Post-condition	Campaign status displayed in dashboard.
Actual Result	As Expected.
Pass/Fail	Pass

Test Case ID	TC020
Test Objective	Verify admin can pause an active campaign.
Description	System pauses an ongoing campaign, stopping any further outbound calls.
Precondition	Active campaign is running with pending calls.
Steps	1. Admin clicks Pause Campaign. 2. System halts further call initiation. 3. Campaign status updated.
Test Data	Campaign ID: C-005, Status: Active → Paused
Expected Result	Campaign paused; no new calls initiated; pending calls retained.
Post-condition	Campaign status updated to Paused in dashboard.
Actual Result	As Expected.
Pass/Fail	Pass

References

- [1] Liesenfeld et al. (2023) – Real-time speech processing for conversational AI. arXiv:2307.15493. <https://arxiv.org/abs/2307.15493>
- [2] Shivakumar et al. (2019) – Intent detection from speech with recognition errors. arXiv:1904.03576. <https://arxiv.org/abs/1904.03576>
- [3] Li et al. (2018) – Dialogue context tracking and understanding. arXiv:1810.09154. <https://arxiv.org/abs/1810.09154>
- [4] Alhussein et al. (2025) – Speech emotion/sentiment detection in conversations. Springer. <https://link.springer.com/article/10.1007/s10462-025-11197-8>
- [5] Twilio Docs (2026) – Programmable Voice API. <https://www.twilio.com/docs/voice>
- [6] ElevenLabs TTS (2026) – Text-to-speech solutions. <https://elevenlabs.io>
- [7] FastAPI Docs (2026) – Asynchronous Python web framework. <https://fastapi.tiangolo.com>
- [8] React (2026) – React.js official documentation. <https://reactjs.org>
- [9] Tailwind CSS (2026) – Utility-first CSS framework. <https://tailwindcss.com>
- [10] Supabase Docs (2026) – Open source backend platform. <https://supabase.com/docs>
- [11] Whisper by OpenAI (2023) – Speech recognition models. <https://openai.com/research/whisper>
- [12] Amazon Web Services (2025) – Cloud hosting and virtual machines. <https://aws.amazon.com>
- [13] Groq (2025) – High-performance AI inference platform. <https://www.groq.com>
- [14] Docker Docs (2026) – Containerised deployment and orchestration. <https://docs.docker.com>
- [15] Li, X., Chen, Y., Wu, J. (2022) – Multi-turn dialogue management for voice assistants. Journal of AI Research, 75(3), 1123–1145.
- [16] Zhang, H., Wang, S. (2021) – Real-time sentiment analysis in conversational AI. IEEE Access, 9, 12345–12357.
- [17] OpenAI ChatGPT API (2026) – Documentation for conversational AI APIs. <https://platform.openai.com/docs>
- [18] Kumar, A., Patel, R. (2020) – Automatic intent recognition for voice bots. International Conference on Speech and AI, 45–52.
- [19] Microsoft Azure Cognitive Services (2025) – Speech-to-text and text-to-speech services. <https://azure.microsoft.com/en-us/services/cognitive-services/speech>
- [20] Twilio Blog (2025) – Voice automation and customer engagement case studies. <https://www.twilio.com/blog/voice-bots>
- [21] Ian Sommerville. Software Engineering. Pearson, 10th edition, 2016.
- [22] Roger S. Pressman and Bruce Maxim. Software Engineering: A Practitioner's Approach. McGraw-Hill Education, 9th edition, 2020.
- [23] Martin Fowler. UML Distilled: A Brief Guide to the Standard Object Modeling Language. Addison-Wesley, 3rd edition, 2018.

Appendices

Appendix A

A.1.1 Use Case Diagrams for Nora – AI Voice Agent System

The use case diagrams below cover all four modules of the Nora system. Each diagram shows the actors interacting with that module and the use cases they trigger. Actors across all diagrams include: Admin, Customer, Human Agent, System (automated/scheduled), and Compliance Engine.

UC Diagram 1 – Module 1: AI Voice Agent

Covers: Initiate Outbound Reminder Call (UC-1.1), Detect and Process User Response (UC-1.2), Escalate Call to Human Agent (UC-1.3), Retry Failed Call (UC-1.4), Terminate Call on Completion (UC-1.5), Enforce Call Limits and Compliance (UC-1.6).

[Figure A.1: Use Case Diagram – Module 1: AI Voice Agent]

UC Diagram 2 – Module 2: Speech Processing

Covers: Convert Speech to Text (UC-2.1), Generate Voice Response from Text (UC-2.2), Detect Language of User Input (UC-2.3), Handle Noise and Improve Recognition Accuracy (UC-2.4).

[Figure A.2: Use Case Diagram – Module 2: Speech Processing]

UC Diagram 3 – Module 3: Conversation Management

Covers: Maintain Conversation Context (UC-3.1), Classify User Intent (UC-3.2), Schedule Follow-Up Reminder (UC-3.3), Detect Sentiment and Adapt Response (UC-3.4).

[Figure A.3: UC Diagram 3 – Module 3: Conversation Management]

UC Diagram 4 – Module 4: Admin Dashboard

Covers: View Call Logs and Transcripts (UC-4.1), View Analytics and Reports (UC-4.2), Export Reports in CSV Format (UC-4.3), Manage User Access and Roles (UC-4.4), Schedule and Manage Reminder Campaigns (UC-4.5).

[Figure A.4: UC Diagram 4 – Module 4: Admin Dashboard]

A.1.2 Detailed Use Case Example

The following illustrates a fully detailed use case for the Initiate Outbound Reminder Call scenario, showing the complete interaction between the Admin, AI Voice Agent Module, Speech Processing Module, Conversation Management Module, and Customer across all flow paths.

[Figure A.5: Detailed Use Case – Initiate Outbound Reminder Call]

A.1.3 Event-Response Table for Nora

The event-response table captures all system-level events, their triggering conditions, and the corresponding system responses across all modules.

[Figure A.6: Event-Response Table for the Nora AI Voice Agent System]

A.1.4 Storyboard – Outbound Reminder Call Flow

The storyboard illustrates the end-to-end journey of a typical outbound reminder call from the customer's perspective, covering call initiation, greeting, payment confirmation, and call closure.

[Figure A.7: Storyboard – Outbound Reminder Call Flow]

Appendix B

B.1 Domain Model for Nora

The domain model captures the key conceptual entities of the Nora system and their relationships, independent of implementation details. Core entities include Customer, Call, Transcript, ConversationState, DetectedIntent, ComplianceRecord, AdminUser, and CallLog.

[Figure A.8: Domain Model for the Nora AI Voice Agent System]

Appendix C

C.1 Box and Line Diagram for Nora

The box and line (high-level context) diagram shows all major system modules and external entities. Internal modules include the AI Voice Agent, Speech Processing, Conversation Management, Admin Dashboard, Compliance Engine, and Data Layer (Supabase). External entities include Customer, Admin, Human Agent, Twilio, Whisper, ElevenLabs, and Groq.

[Figure A.9: Box and Line Diagram for the Nora AI Voice Agent System]

C.2 Architecture Pattern for Nora

The multi-tier architecture diagram shows the logical decomposition across three layers: Presentation (Next.js + React + Tailwind CSS), Business Logic (FastAPI, Voice Agent Engine, Speech Processing Service, Conversation Management, Compliance Engine, Intent Detection), and Data & Infrastructure (Supabase, Twilio, Whisper, ElevenLabs, Groq, Docker, Cloud VMs).

[Figure A.10: Architecture Pattern for the Nora AI Voice Agent System]

Appendix D

D.1 Activity Diagram

The activity diagrams for Nora capture the dynamic workflow of an outbound reminder call across six swimlanes: Admin, AI Voice Agent Module, Speech Processing Module, Conversation Management Module, Customer, and Human Agent. The workflow is divided into multiple diagrams to clearly illustrate decision points, retry logic, compliance checks, and escalation paths.

[Figures A.11–A.16: Activity Diagram for the Nora AI Voice Agent System – Parts 1–6]

D.2 Class Diagram

The class diagram defines the ten core system classes, their attributes and methods, and their associations: Customer, Call, Transcript, Utterance, ConversationState, DetectedIntent, SpeechProcessor, AdminUser, CallLog, and ComplianceEngine.

[Figure A.17: Class Diagram for the Nora AI Voice Agent System]

D.3 Sequence Diagrams

The class-level sequence diagrams show the full message exchange between system objects during a complete outbound reminder call session. The workflow is divided into multiple diagrams to clearly represent the payment confirmation path and the escalation alternative fragment within the Nora AI Voice Agent system.

[Figures A.18–A.22: Sequence Diagram for the Nora AI Voice Agent System – Parts 1–5]

D.4 State Transition Diagram

The state transition diagram models the lifecycle of a Call session through all system states: Idle, Initiated, Connected, No Answer, Awaiting Response, Processing Intent, Retrying, Payment Confirmed, Follow-Up Scheduled, Escalated, Closed, and Failed.

[Figure A.23: State Transition Diagram for the Nora AI Voice Agent System]

D.5 Data Flow Diagram

The Data Flow Diagrams (DFDs) at three levels – Level 0 (Context), Level 1 (Process Decomposition), and Level 2 (Sub-process Detail) – illustrate how data moves between external entities (Customer, Admin, Human Agent), system processes (Call Initiation, Speech Processing, Intent Detection, Conversation Management, Logging), and data stores (D1: Customer Records, D2: Call Logs, D3: Transcripts & Intents).

DFD Symbols and Guidelines: The DFDs use Gane-Sarson notation: rectangles represent external entities; rounded rectangles represent processes; arrows represent data flows; open rectangles represent data stores.

[Figure A.24: Data Flow Diagram – Level 0 (Context Diagram)]

[Figure A.25: Data Flow Diagram – Level 1 (Process Decomposition)]

[Figure A.26: Data Flow Diagram – Level 2 (Sub-process Detail)]

